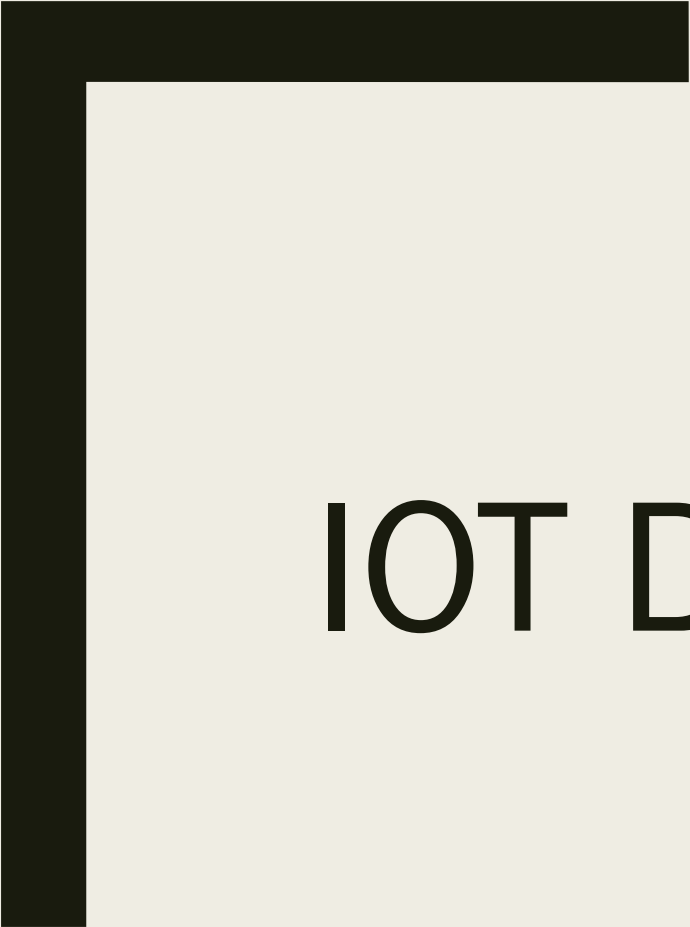
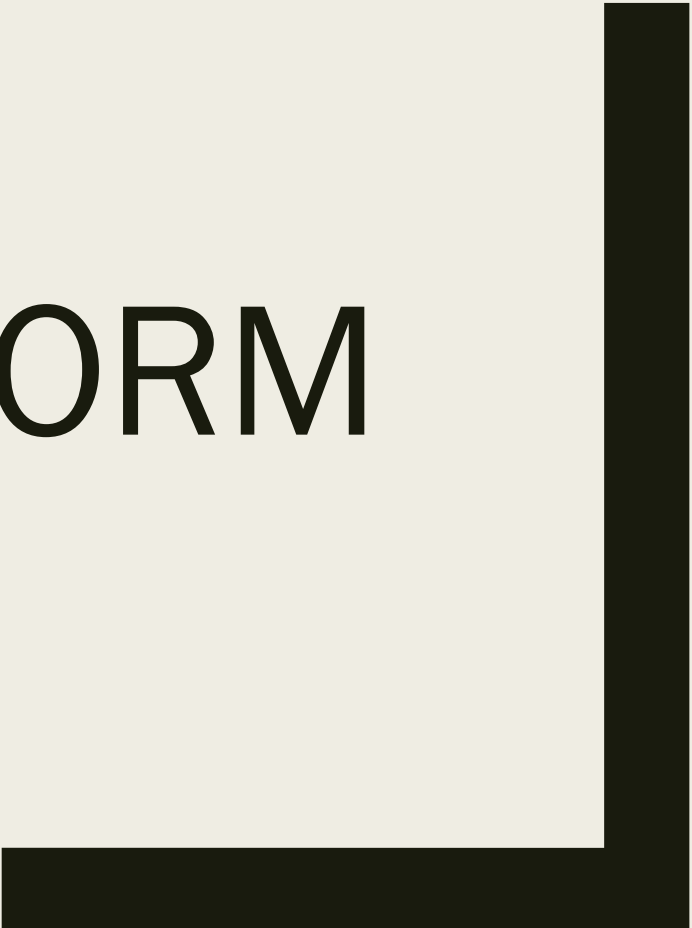




IOT DATA PLATFORM



Цель проекта

- Создать универсальную платформу обработки данных для IoT-систем:
 - *Сбор данных*
 - *Потоковая обработка*
 - *Хранение данных*
 - *Предоставление информации пользователям*

Проблема

- Сегодня существует множество IoT-проектов
- Небольшой команде сложно собрать экспертов во всех областях:
 - *Системы сбора данных*
 - *Потоковая обработка*
 - *Хранение, предоставления данных*
 - *Мониторинг и развертывание*
- Изучение всех инструментов и их интеграция занимают **очень много времени**

Технологии

- Мы объединили готовые open source системы в одну платформу:
 - *Kafka* – прием входящих данных
 - *Flink* – потоковая обработка
 - *Redis* – быстрые низколатентные чтения
 - *Trino* – SQL-движок для интерактивной аналитики по данным в PostgreSQL, Iceberg, S3
 - *PostgreSQL* – реляционные данные
 - *S3-совместимое хранилище* – исторические данные
 - *Apache Iceberg* – табличный слой поверх data lake
 - *Kubernetes / Docker Compose / Helm* - развертывание

Прогресс (до января)

■ 3 Проекта-примера

- *Автобусы: обработка прибытий*
- *Датчики: оконные агрегации и подсчет сухих дней*
- *Rich detector: фильтрация колец*

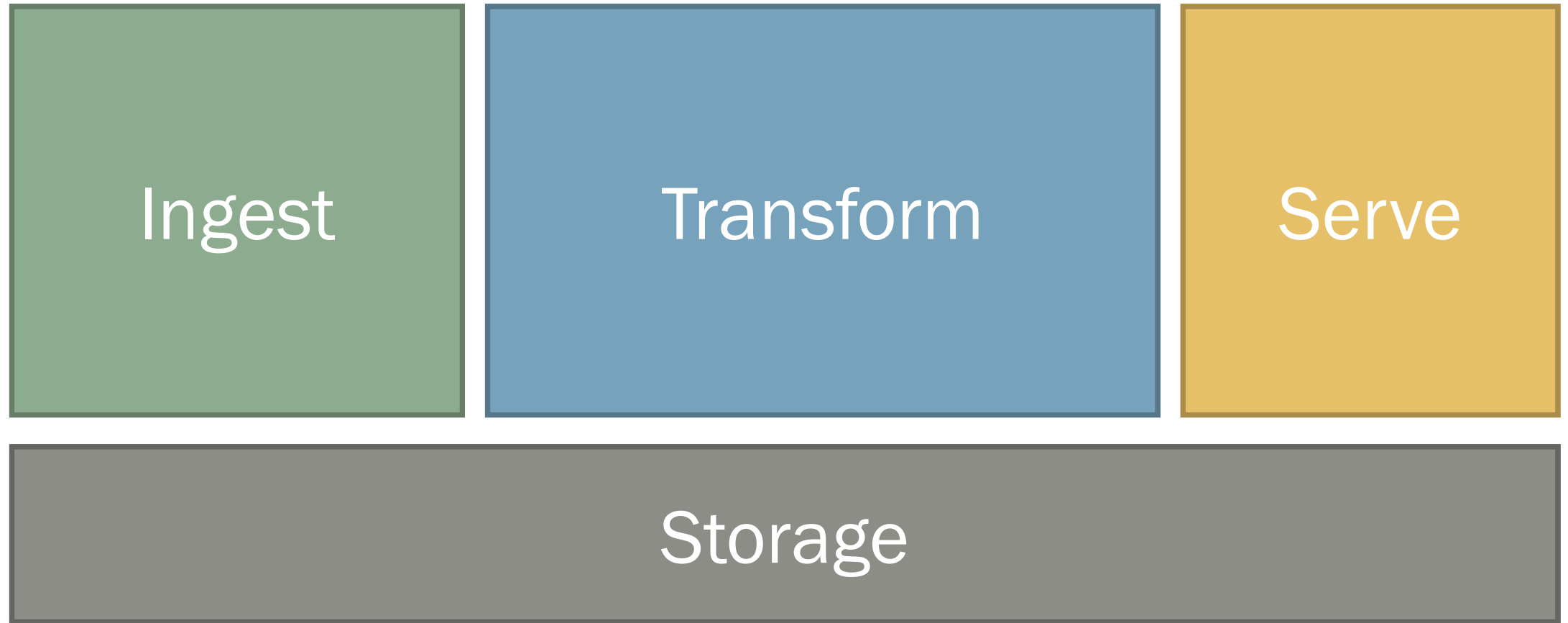
■ Разработка платформы

- *Изучили технологии*
- *Проработали архитектуру и выбрали стек*
- *Интегрировали компоненты и собрали базовый data-pipeline*
- *Реализовали конфигурирование*

Прогресс (с февраля)

- Доработан демо-проект PVZ:
 - *Переход с PostgreSQL на TimescaleDB.*
- Доработан демо-проект Cherenkov Detector:
 - *Добавлен дамп данных в PostgreSQL.*
- Протестирована платформа:
 - *Проведены latency-замеры на примере демо-проектов PVZ, AVS.*
- Реализован MVP – constructor
 - *Подготовлен план релиза*
 - *Реализован демо-проект AVS*
 - *Написана статья на Habr*

IoT Data Platform





1. Установите git-lfs, если ещё не установлен

```
curl -s https://packagecloud.io/install/repositories/github/git-lfs/script.deb.sh | sudo bash  
sudo apt install git-lfs  
git lfs install
```



2. Склонируйте репозиторий

```
git clone https://github.com/IoTDataPlatform/constructor.git  
cd constructor
```



3. Запустите конструктор

```
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags MQTT,POSTGRES,FLINK,REDIS)
```



В --tags перечисляются компоненты для конструирования, выберите любой набор. Сейчас доступны:

- **MQTT**, MQTT-брокер и эмулятор данных от устройств
- **SOURCE**, эмулятор данных от устройств в виде коннектора
- **STREAMS**, приложение потоковой обработки на Kafka Streams
- **POSTGRES**, БД PostgreSQL
- **ICEBERG**, Lakehouse на S3/Iceberg
- **TRINO**, федеративный движок SQL запросов Trino
- **FLINK**, потоковая обработка на Flink
- **REDIS**, in-memory кэш на Redis

4. Запустите систему

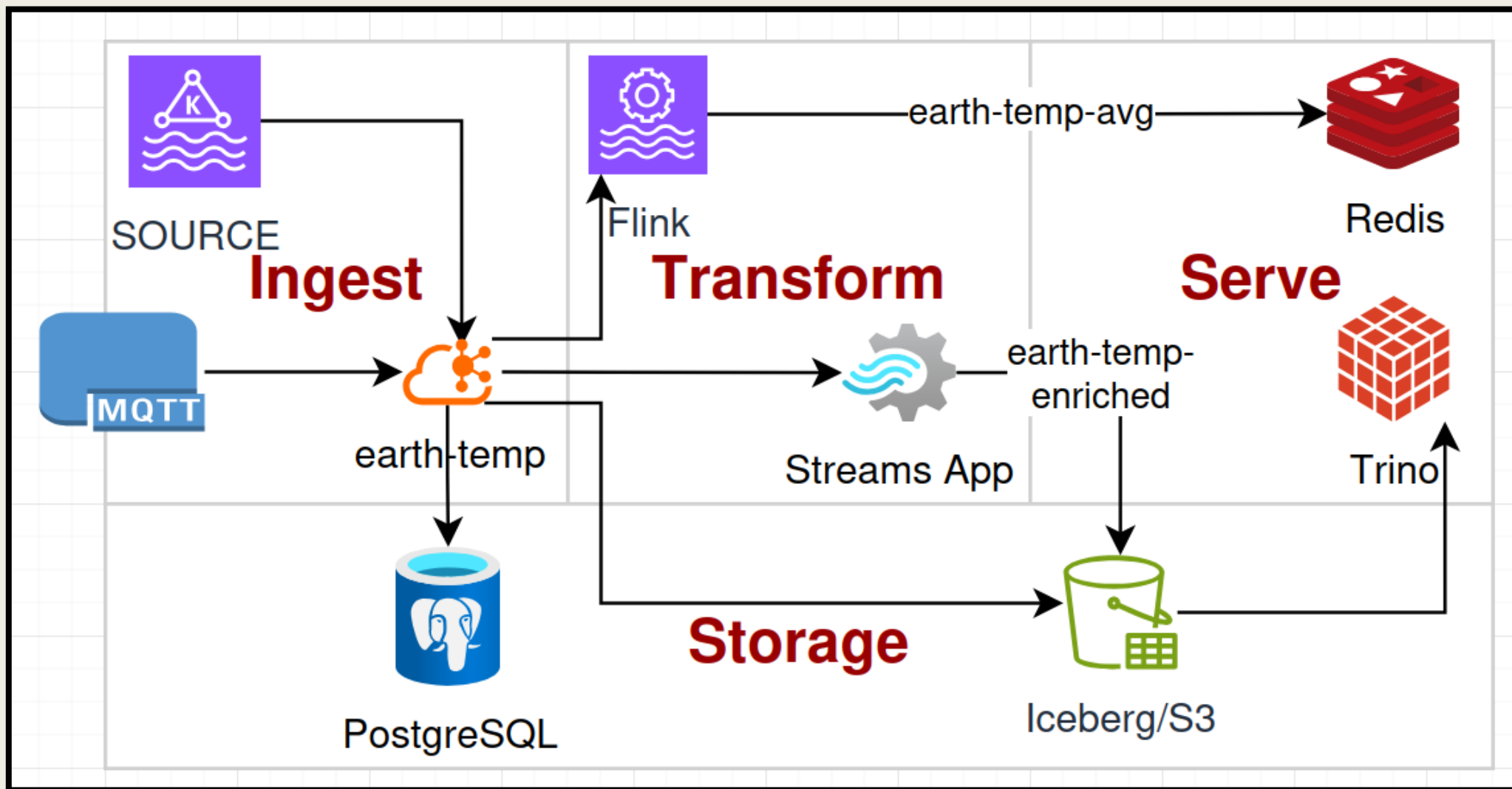
```
docker compose up -d --build
```



5. Остановите систему и почистите директорию

```
docker compose down -v  
git clean -fd
```







mirongaskov 21 апр в 08:37

Современный дата-стек: потоковая система из «LEGO»

🔖 Средний

🕒 17 мин

👁 7.7K

Big Data*, DevOps*, Java*, Распределённые системы*

Тutorial

Читатели

1.6K

1.6K просмотров,
47 повторов

Дочтения

7.2%

Дочитали статью
до конца

Доскроллы

17.3%

Просмотрели статью
до конца



Demo PVZ

За последние 10 минут:

5 датчика

5 онлайн

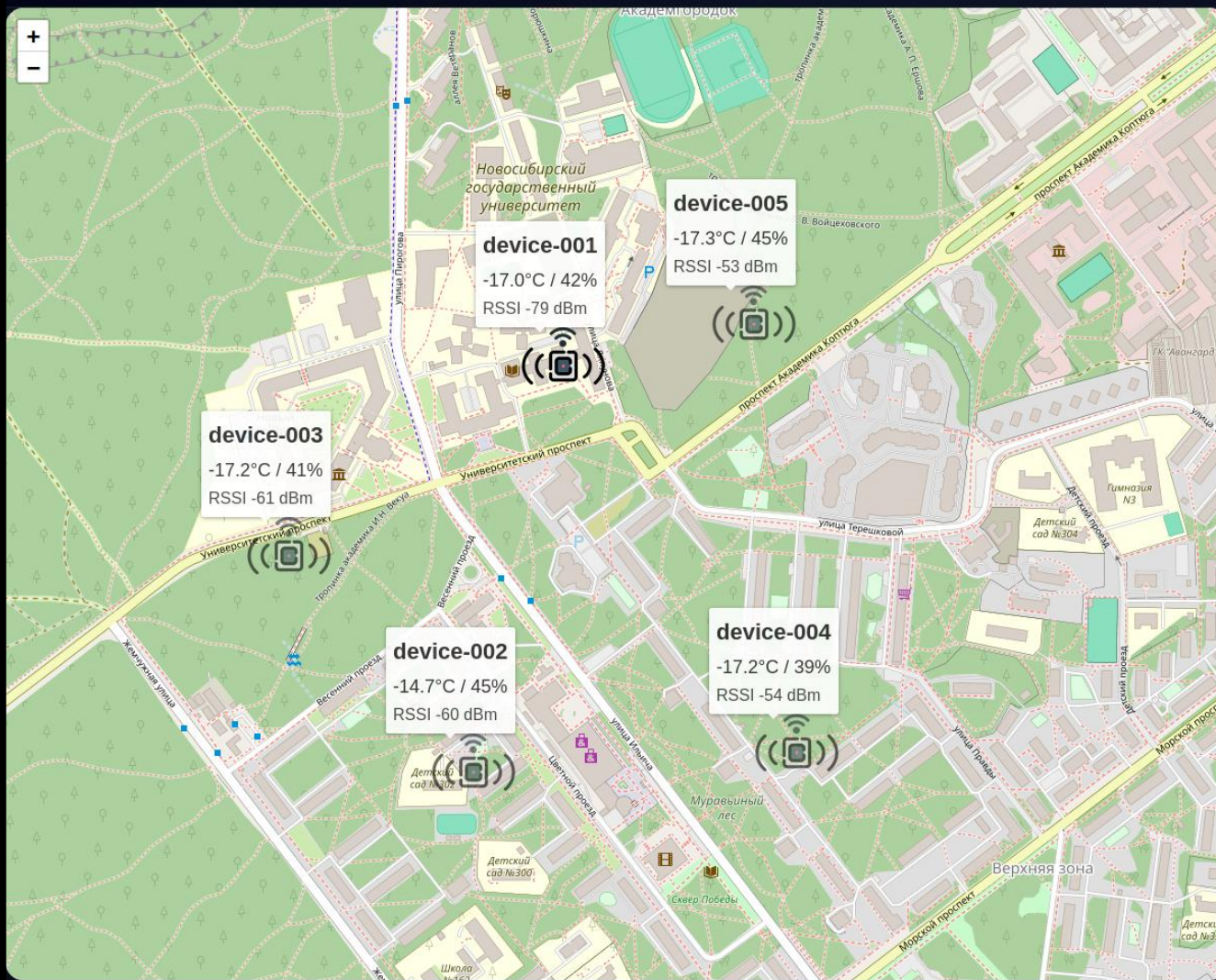
0 офлайн

ср. Т -16.2°C

ср. Н 45%

в засухе 5

макс засуха 1.95д



Устройство device-001

prod / tenant-1

Влажность: 42.0 %

Температура: -17.0 °C

Онлайн: да

Батарея: 76.90 В

RSSI: -79 dBm

SNR: 2.4 dB

Последние данные (h/t): 8/31/57931, 5:00:53 AM

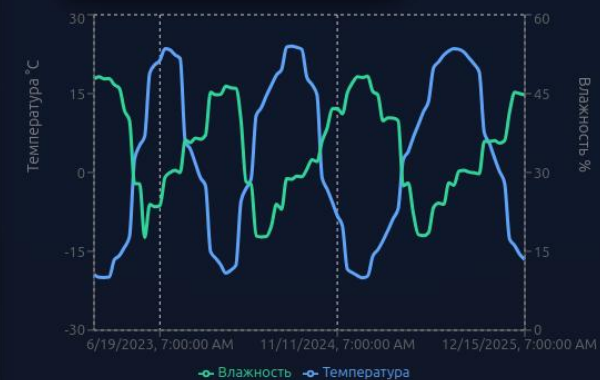
Последнее состояние: 8/31/57931, 5:00:53 AM

Температура / Влажность

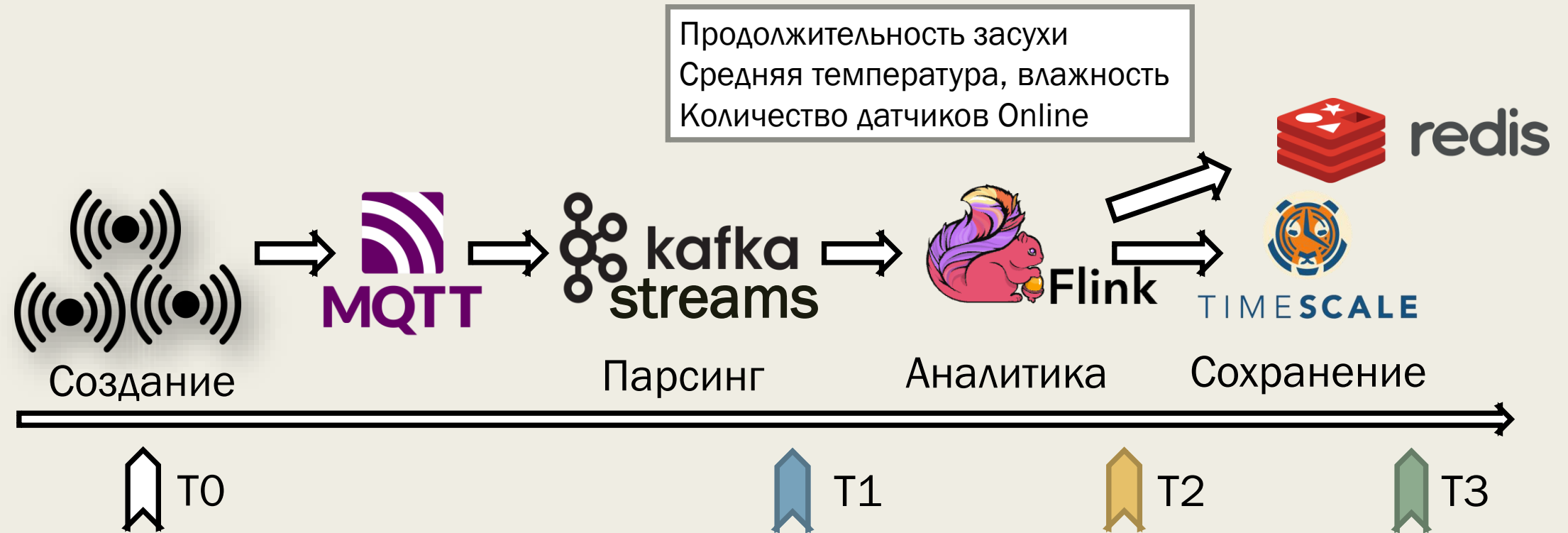
Агрегация: Недели

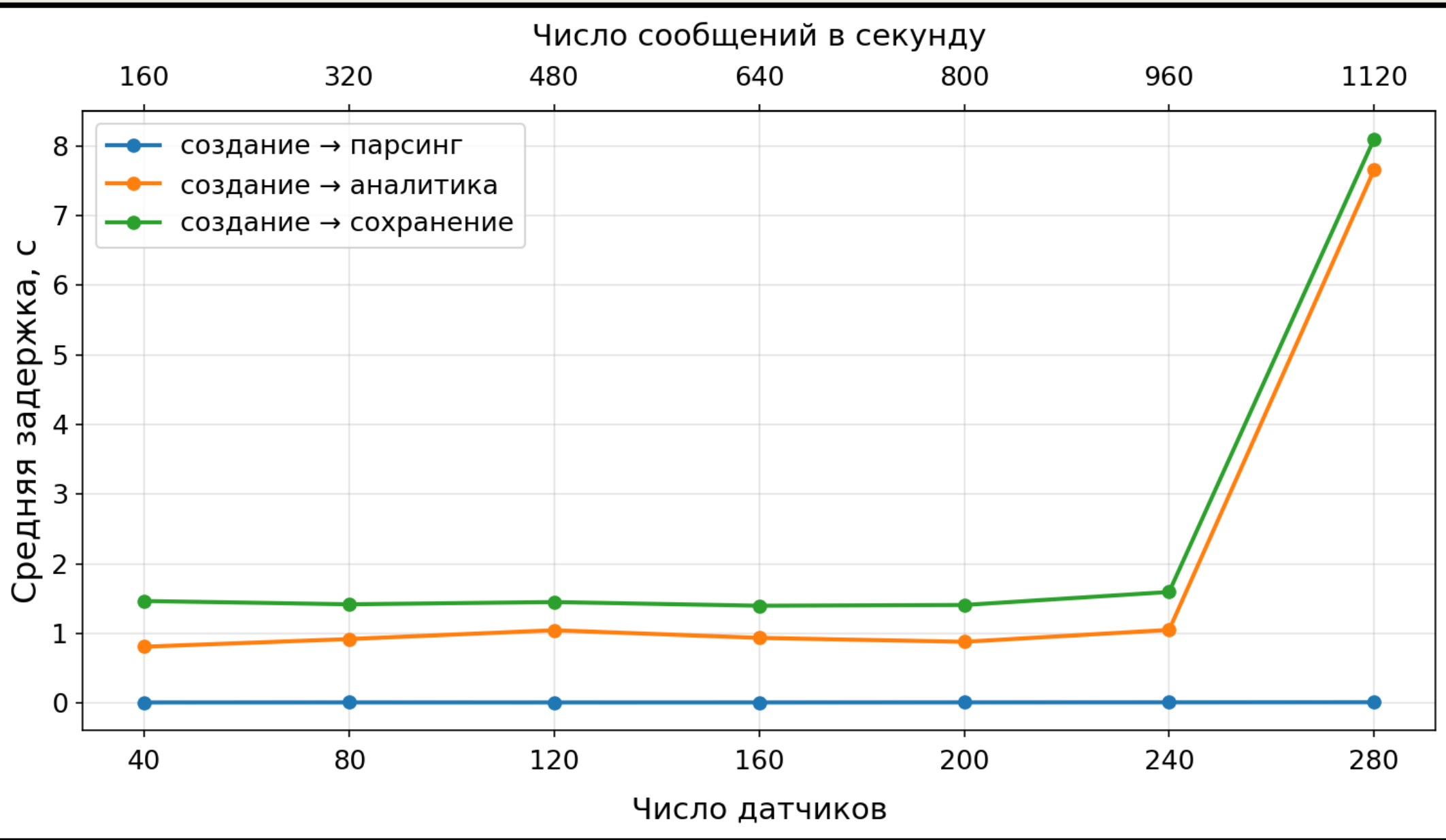
От: 12 / 17 / 2022 , 08 : 50 PM

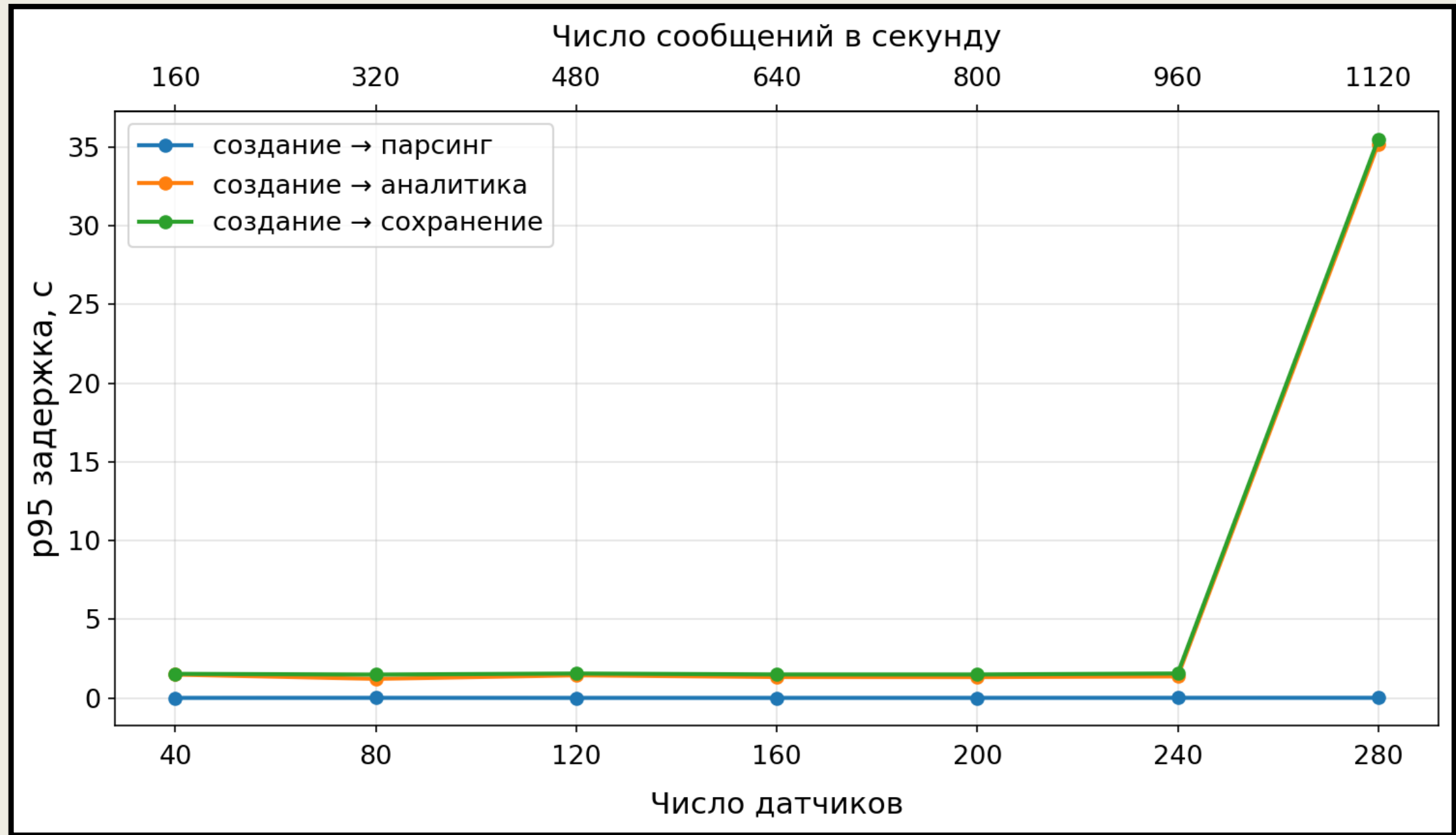
До: 12 / 17 / 2025 , 08 : 50 PM

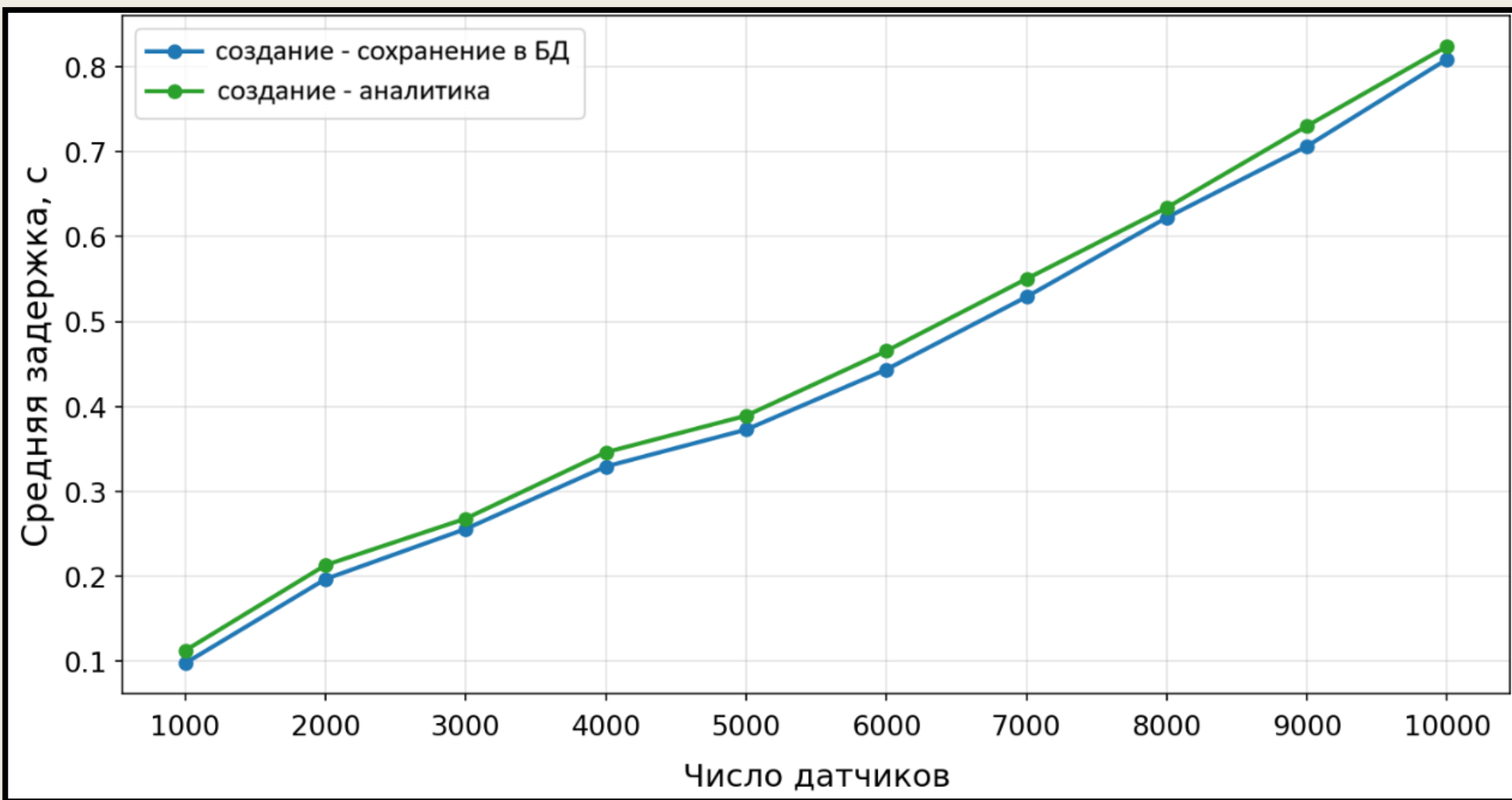


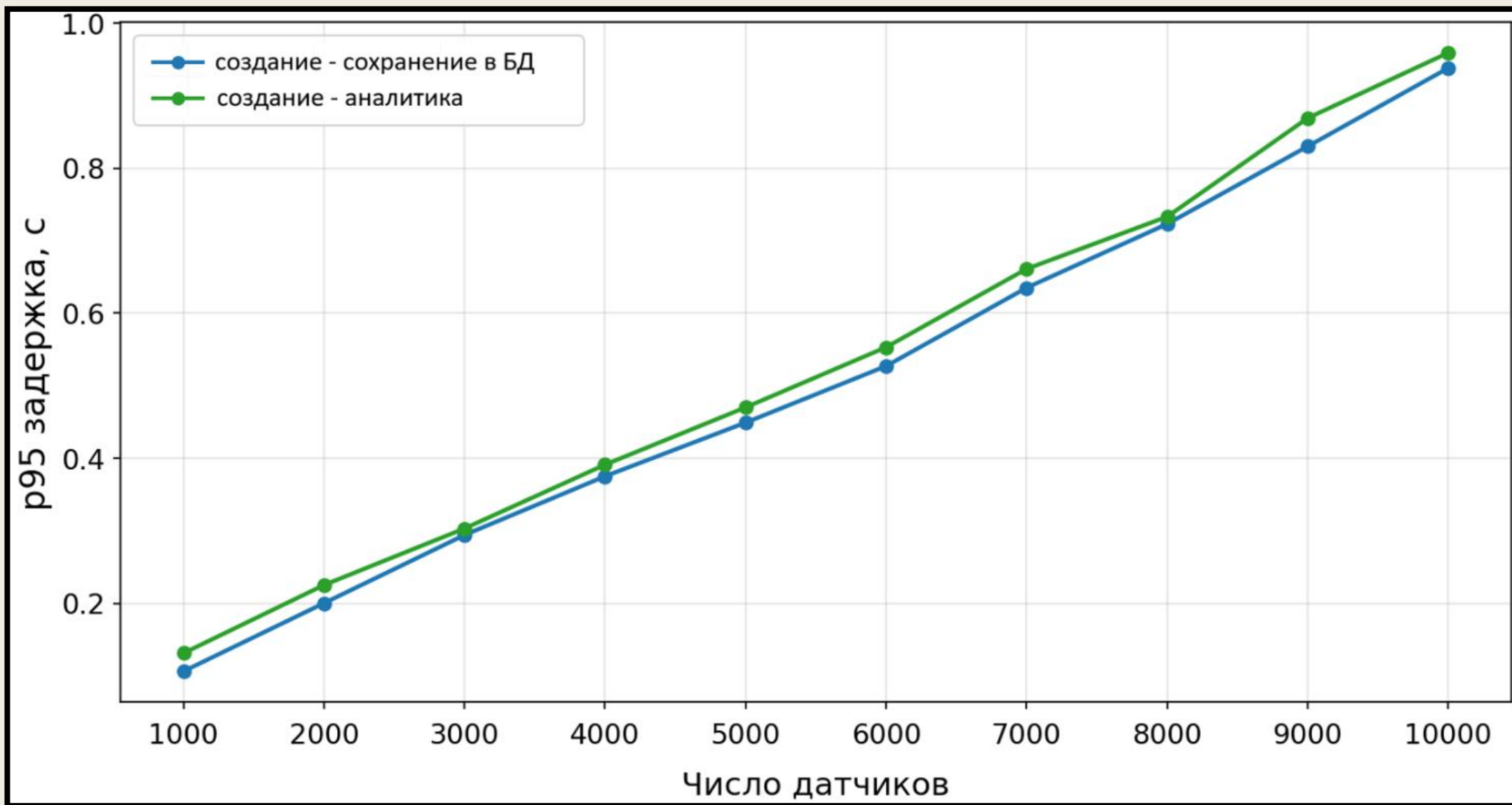
Message flow











Demo AVS

AVS Dashboard

Комнаты, датчики, статус воздуха и переход к подробной аналитике.

Работающие сенсоры

400

Всего замечено: 400

Комнаты в работе

397

Обновлено: 26.04.2026, 14:02:30

CO₂ warning

98

CO₂ critical

0

Temperature warning

50

Temperature critical

2

Humidity warning

0

Humidity critical

0

Настройки витрины

Период для комнат

Последнее значение ▾

Период для датчиков

Последнее значение ▾

Здание

Все здания ▾

Статусы датчиков



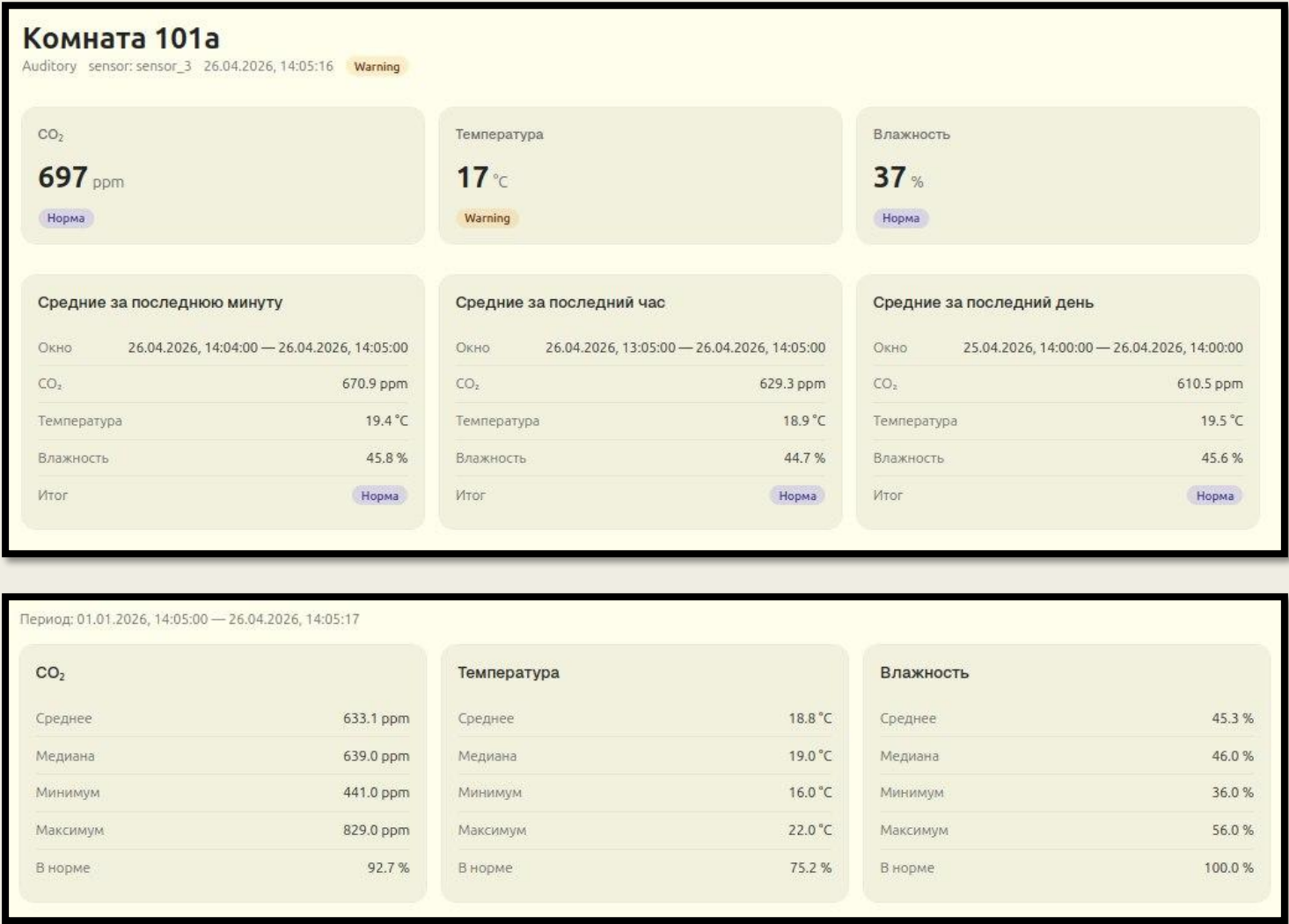
Отлично Норма Warning
Плохой воздух

Статусы комнат



Отлично Норма Warning
Плохой воздух

Demo AVS



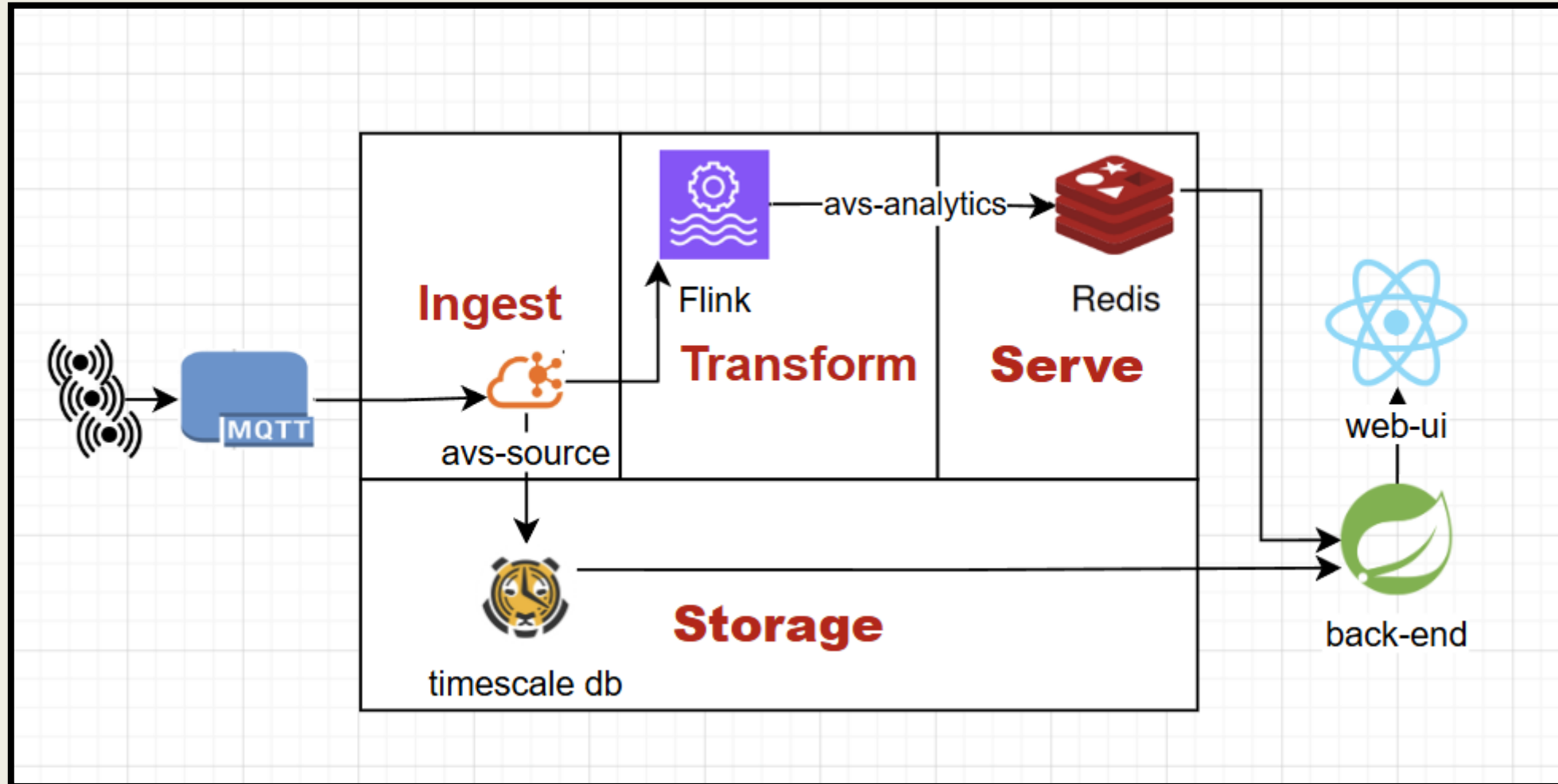
Demo AVS



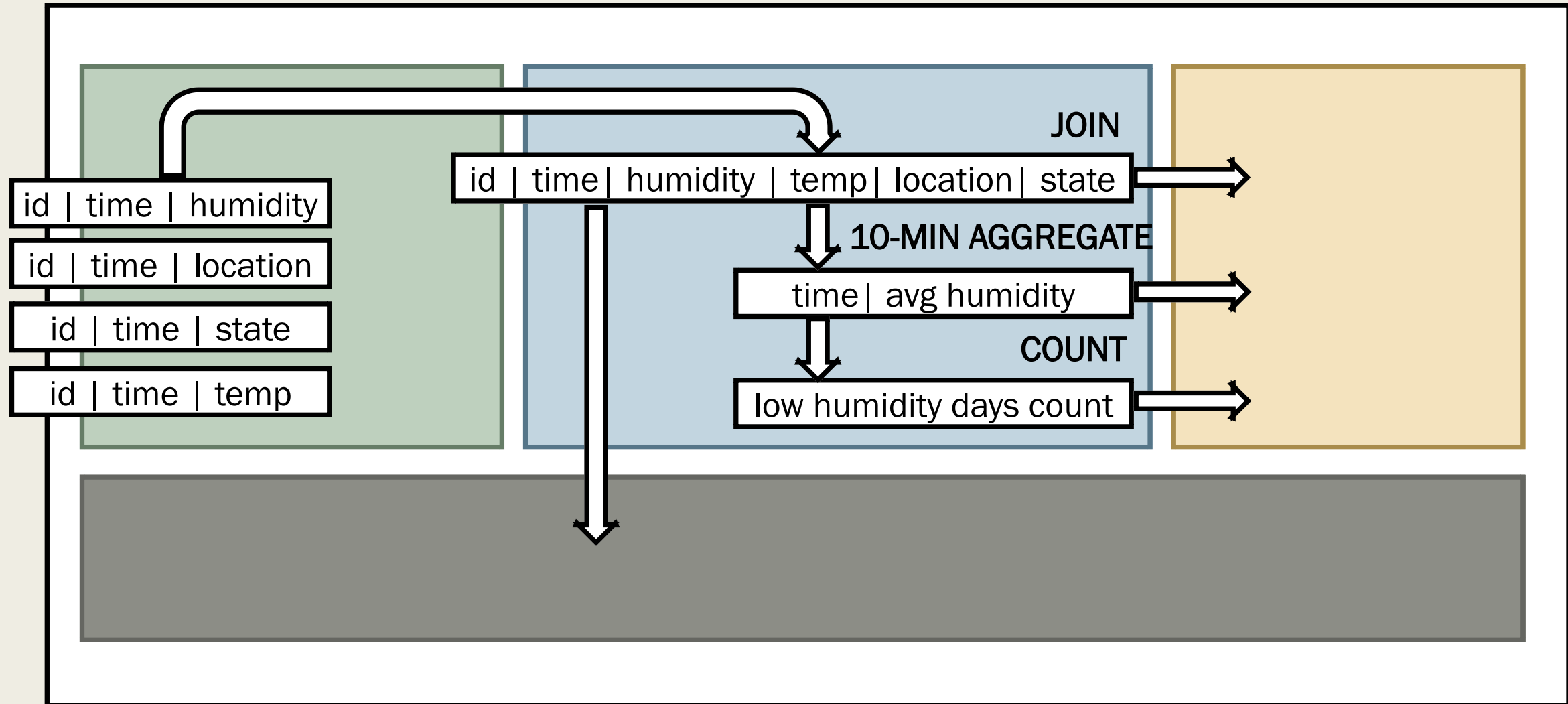
Demo AVS

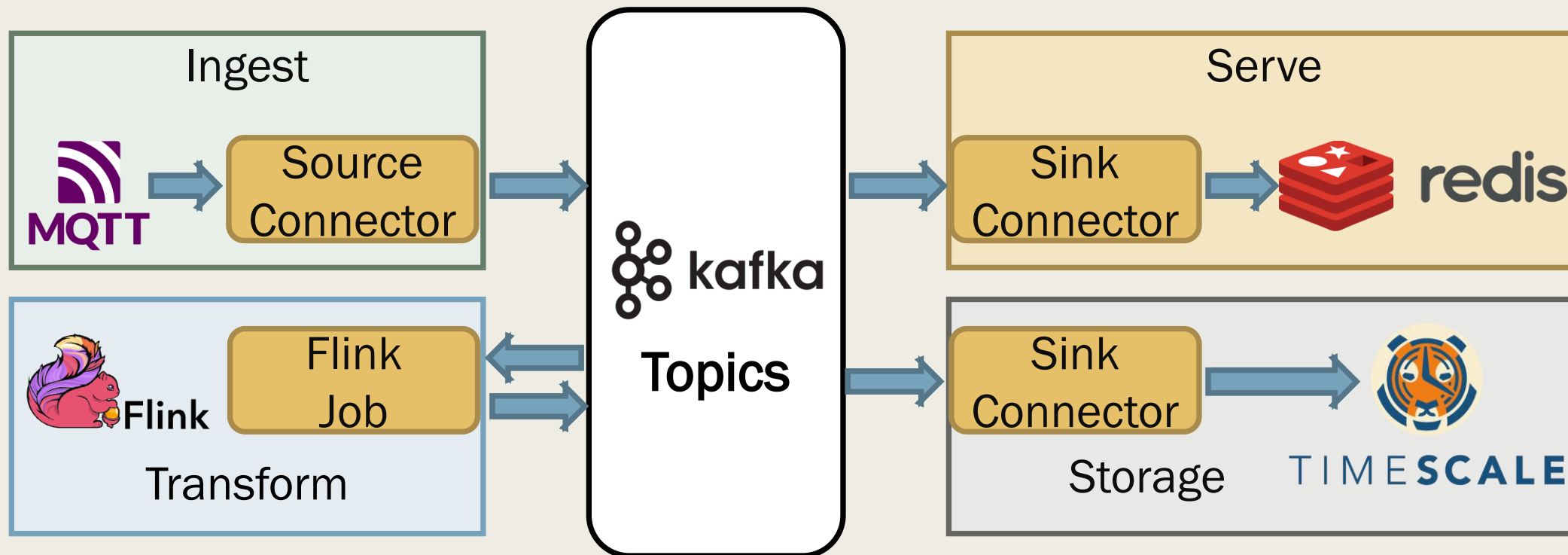
- Аналитика на Flink:
 - *Общая информация по всем датчикам:*
 - Число работающих сенсоров;
 - Число комнат с “отличным, нормальным и плохим воздухом”;
 - *По конкретному датчику и комнате:*
 - Средние показатели за последнюю минуту, час, день
 - Характеристика “влажности, температуры, углекислого газа”
- Timescale DB:
 - *Используется для построения графиков*

Demo AVS

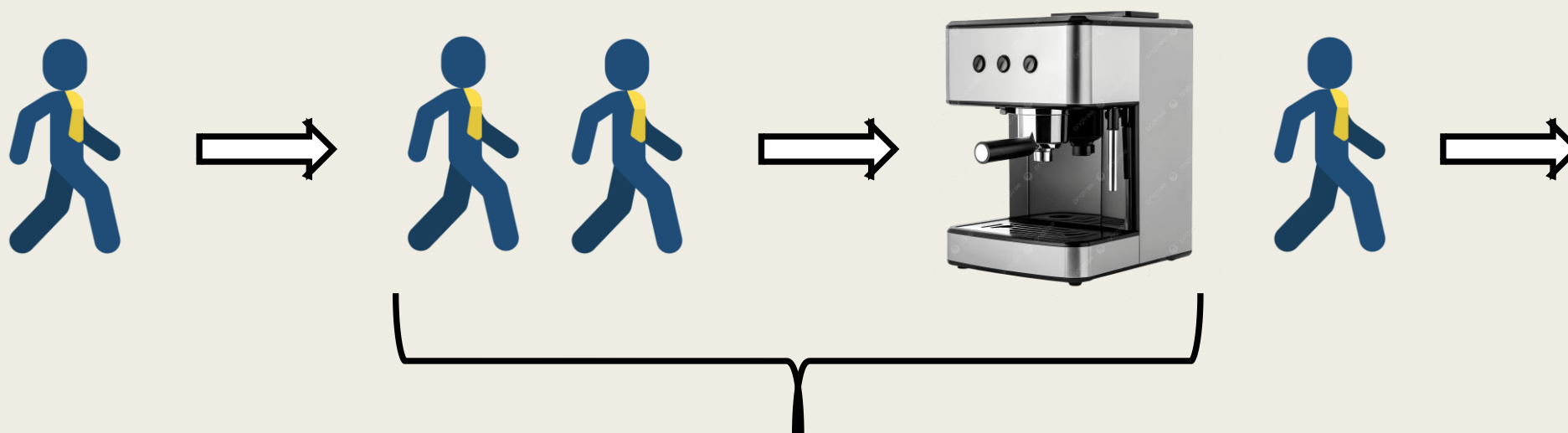


Demo PVZ (Lineage)



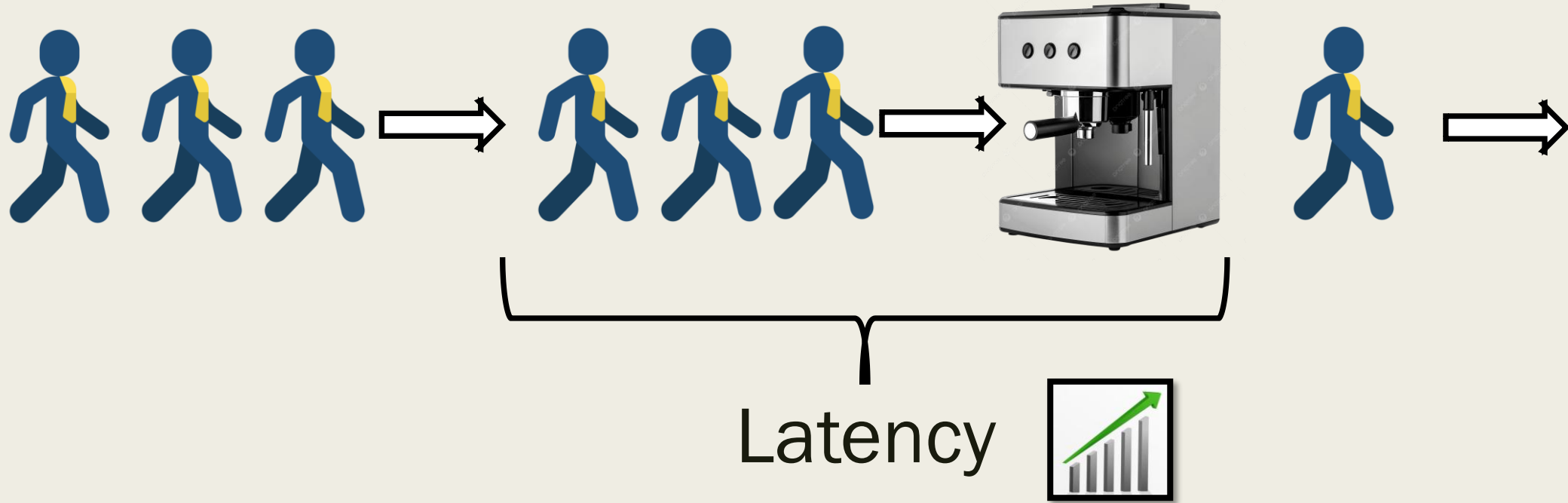


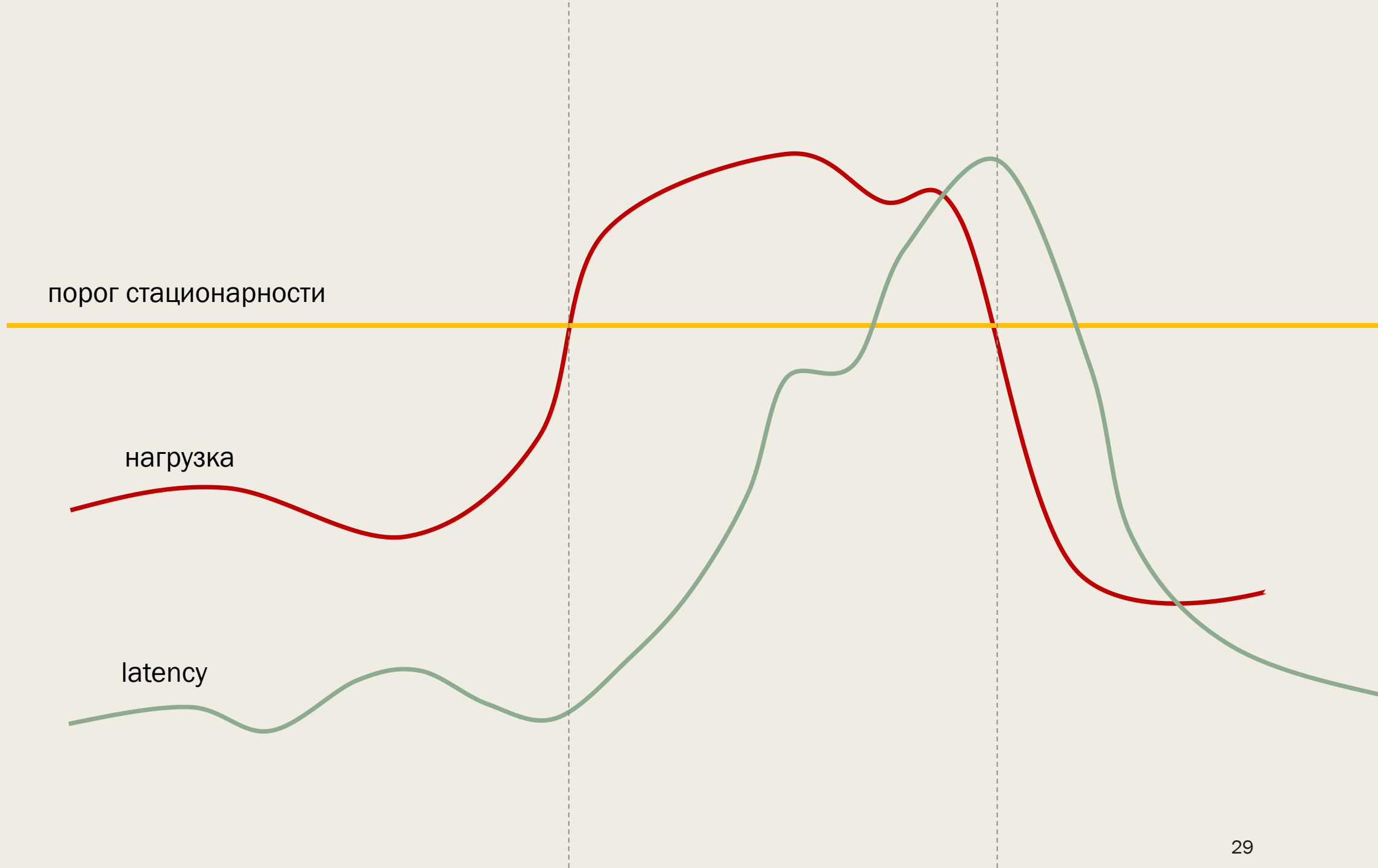
Стационарная система



Latency = Const

Нестационарная система

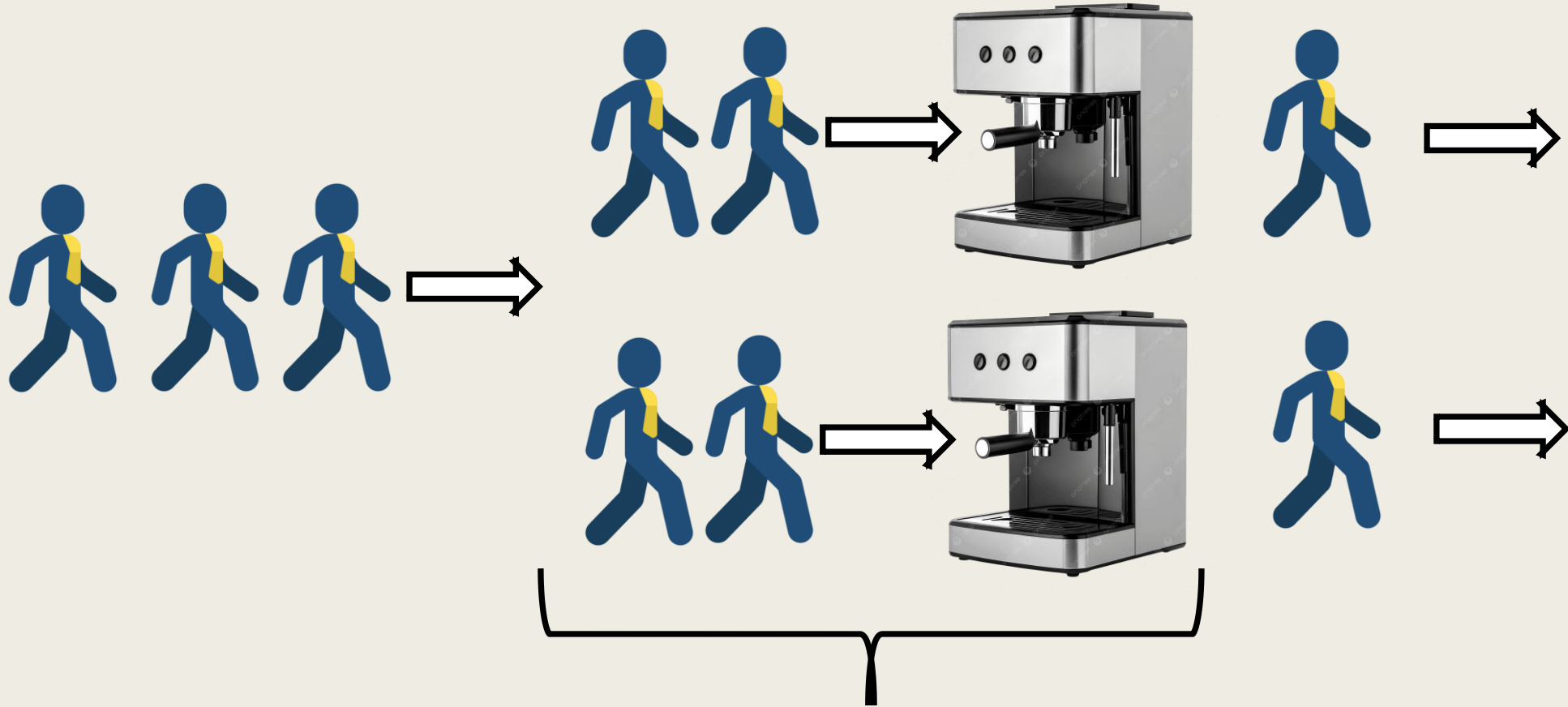




Способы борьбы с latency при увеличении числа датчиков

- Увеличить интервал отправки сообщений датчиком
 - сообщения о влажности почвы не имеет смысла отправлять каждую секунду
- Оптимизировать вычислительный пайплайн
 - переход от декларативных *Flink SQL job* к реализации на *Java*
- Распараллелить поток сообщений на несколько разных узлов





Latency = Const

Batch Processing

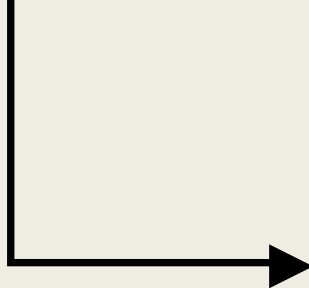
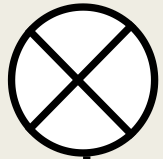
Gauge



id	time	temperature
1	10:00	16
2	10:01	20
1	10:05	25
2	10:10	24
2	10:11	23
2	10:12	24
1	10:12	25

Batch Processing

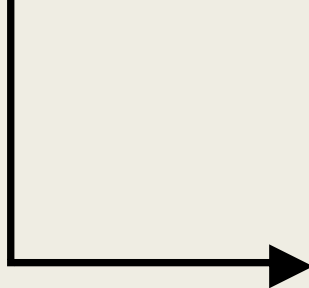
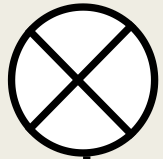
Gauge



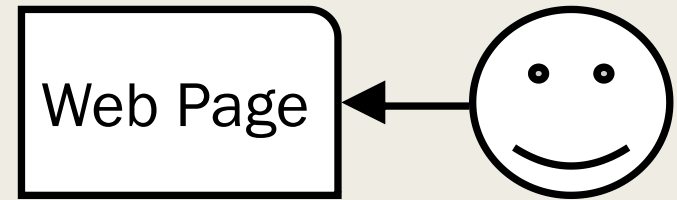
id	time	temperature
1	10:00	16
2	10:01	20
1	10:05	25
2	10:10	24
2	10:11	23
2	10:12	24
1	10:12	25
2	10:15	23

Batch Processing

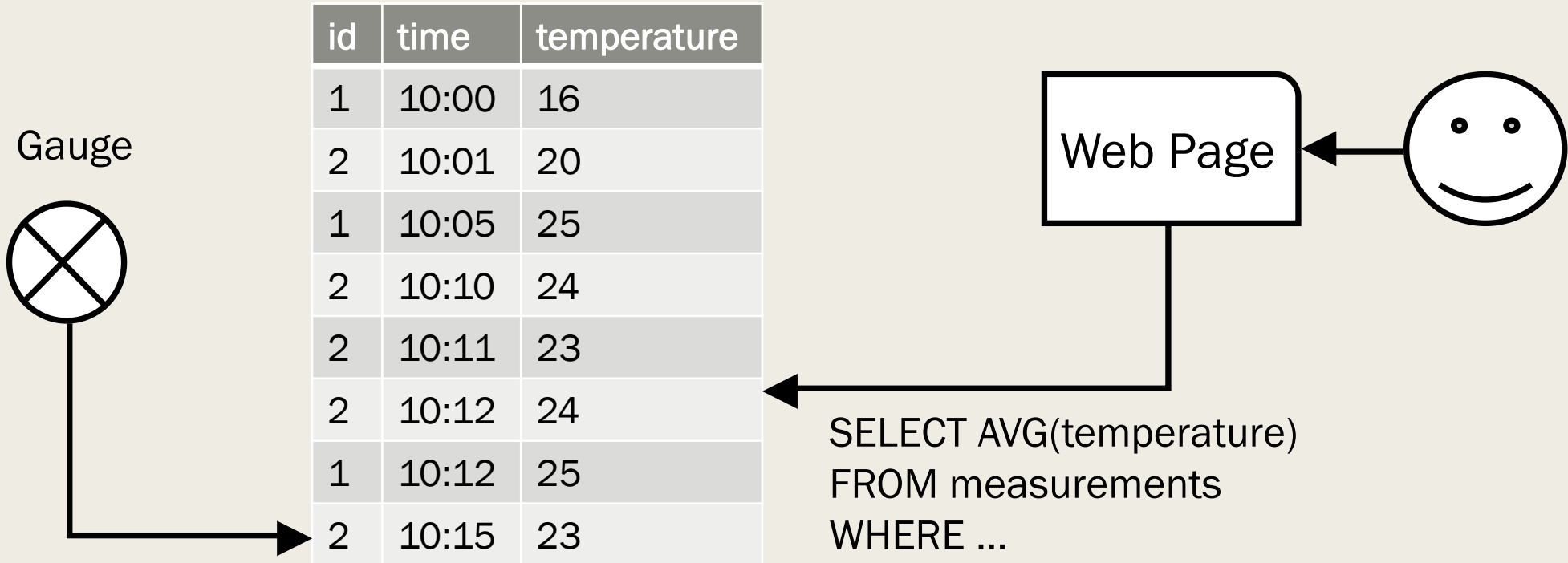
Gauge



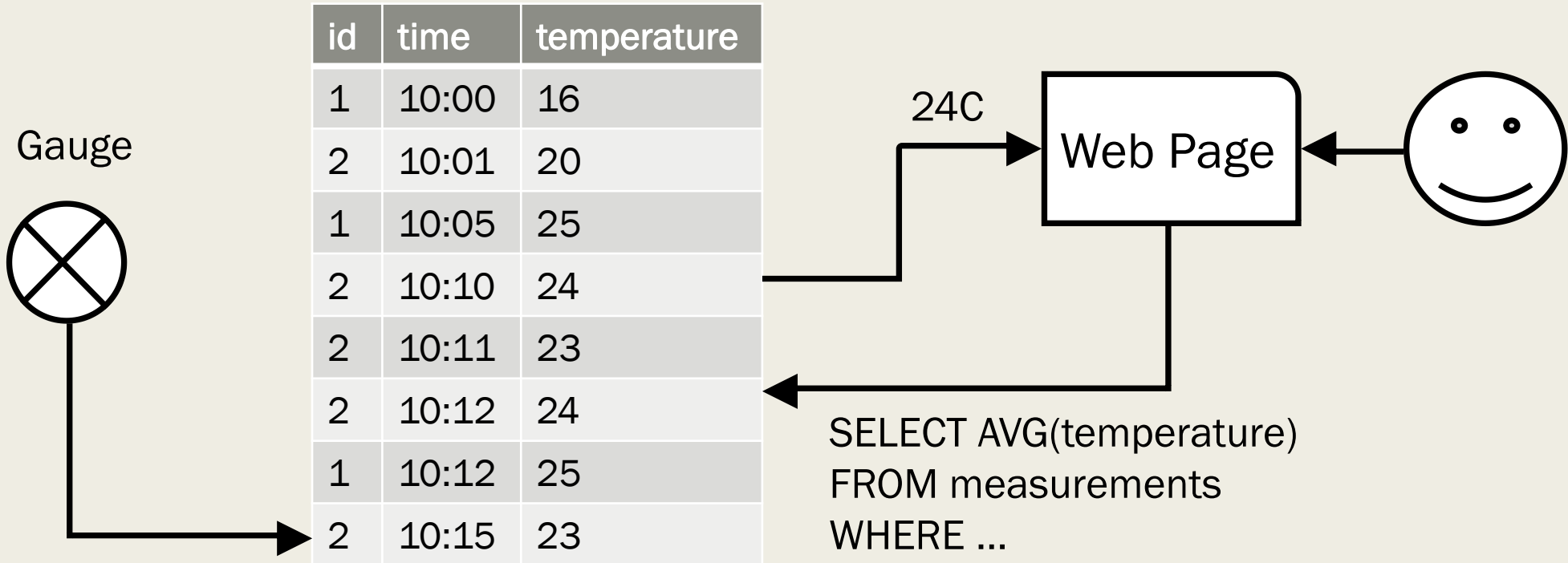
id	time	temperature
1	10:00	16
2	10:01	20
1	10:05	25
2	10:10	24
2	10:11	23
2	10:12	24
1	10:12	25
2	10:15	23



Batch Processing



Batch Processing

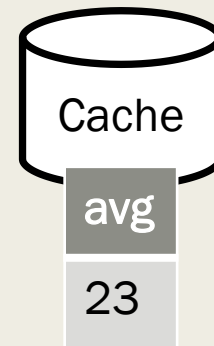


Stream Processing

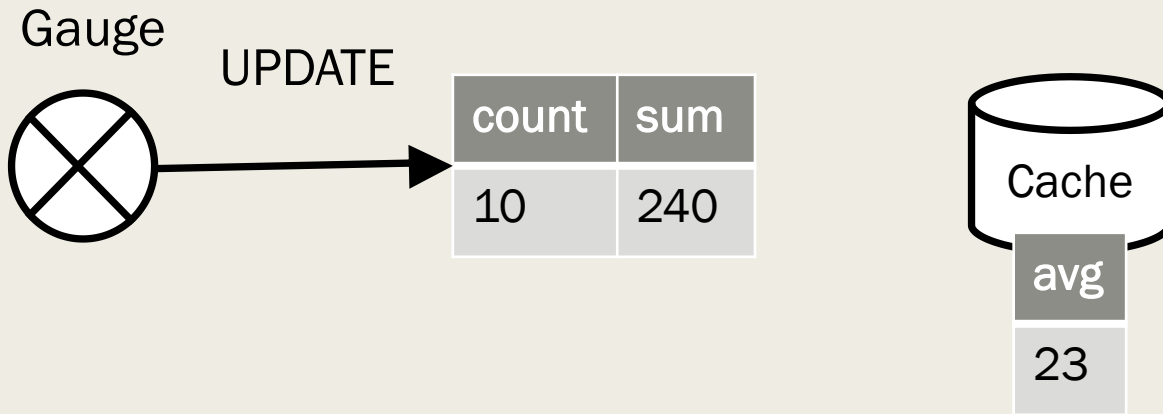
Gauge



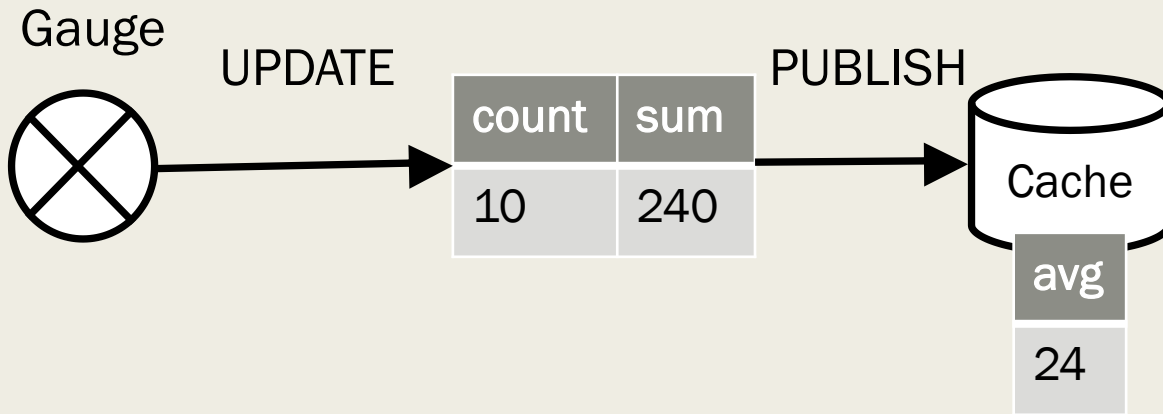
count	sum
9	207



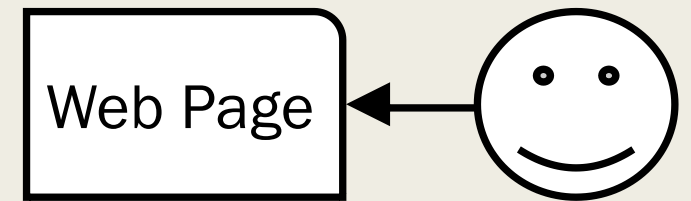
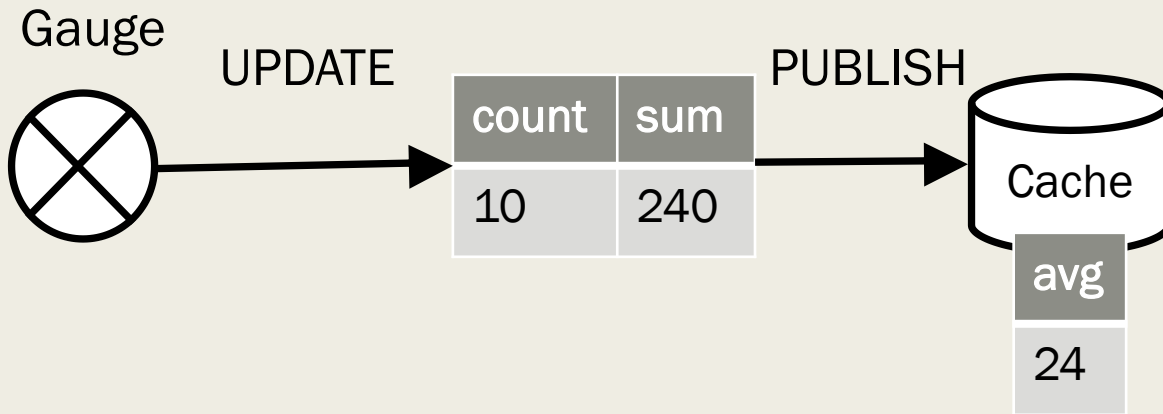
Stream Processing



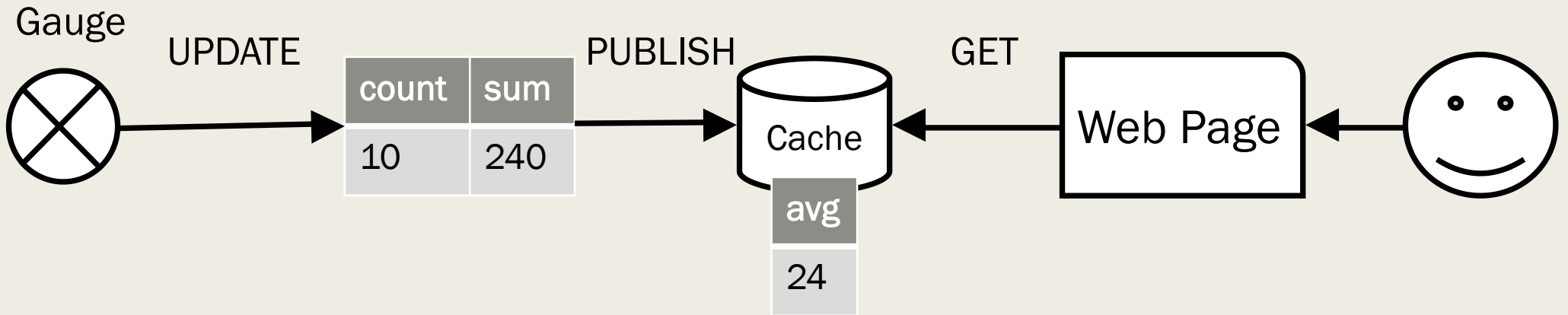
Stream Processing



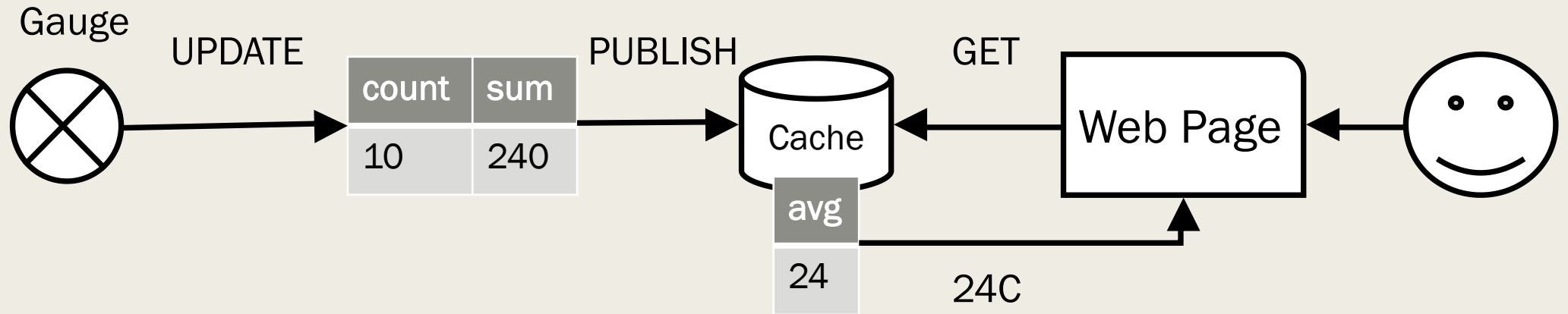
Stream Processing

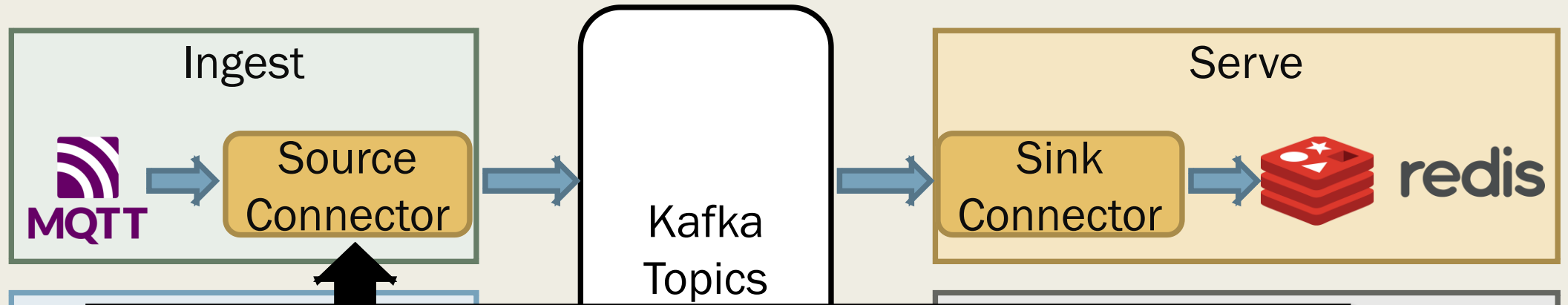


Stream Processing



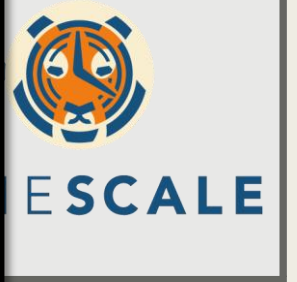
Stream Processing

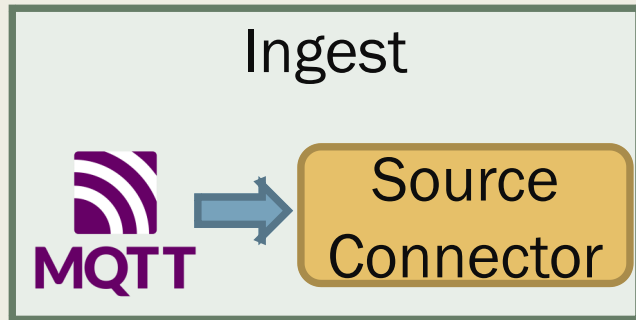




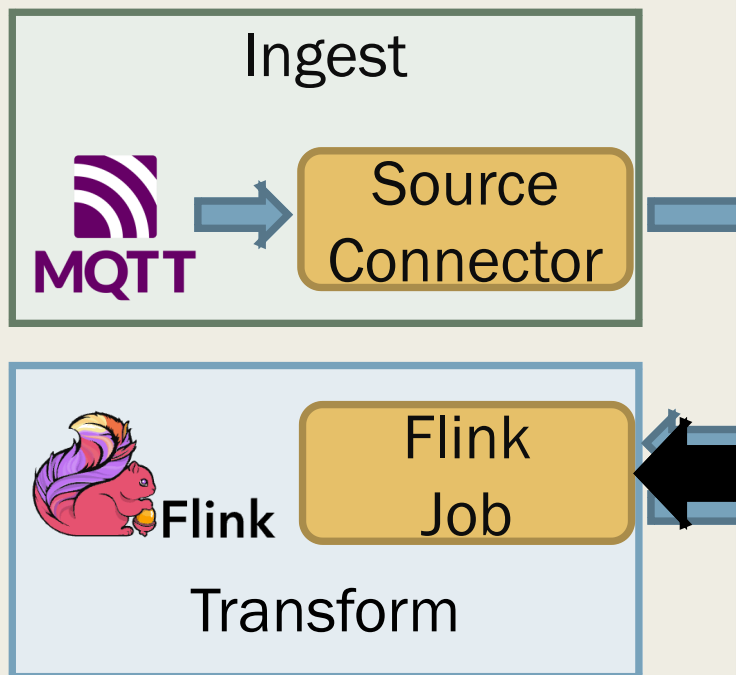
```
{
  "name": "mqtt-sensors-raw-source",
  "config": {
    "connector.class": "com.datamountaineer.streamreactor.connect.mqtt.source.MqttSourceConnector",
    "connect.mqtt.hosts": "tcp://mqtt-broker:1883",
    "connect.mqtt.client.id": "mqtt-sensors-raw-source-1",

    "connect.mqtt.kcql":
    "INSERT INTO mqtt_humidity_raw SELECT * FROM +/+/sensors+/+/humidity;
    INSERT INTO mqtt_temperature_raw SELECT * FROM +/+/sensors+/+/temperature;
    INSERT INTO mqtt_location_raw SELECT * FROM +/+/sensors+/+/location;
    INSERT INTO mqtt_state_raw SELECT * FROM +/+/sensors+/+/state"
  }
}
```





```
INSERT INTO mqtt_enriched
SELECT h.d AS d,
       'prod' AS env,
       'tenant-1' AS tenant,
       l.lat AS lat,
       l.lon AS lon,
       h.h AS h,
       h.t AS t,
       h.ts AS ts_ht,
       s.rssi AS rssi,
       s.snr AS snr,
       s.bat AS bat,
       s.online AS online,
       s.ts AS ts_state,
       CONCAT('pvz:prod:tenant-1:device:', h.d, ':state') AS redis_key
FROM mqtt_humidity h
      LEFT JOIN mqtt_location_dim FOR SYSTEM_TIME AS OF h.rt AS l ON h.d = l.d
      LEFT JOIN mqtt_state_dim FOR SYSTEM_TIME AS OF h.rt AS s ON h.d = s.d;
```



```

INSERT INTO mqtt_recent_summary
SELECT CONCAT(env, ':', tenant) AS redis_key,
       env, AS tenantId,
       window_start,
       window_end,
       CAST(600 AS BIGINT) AS windowSeconds,

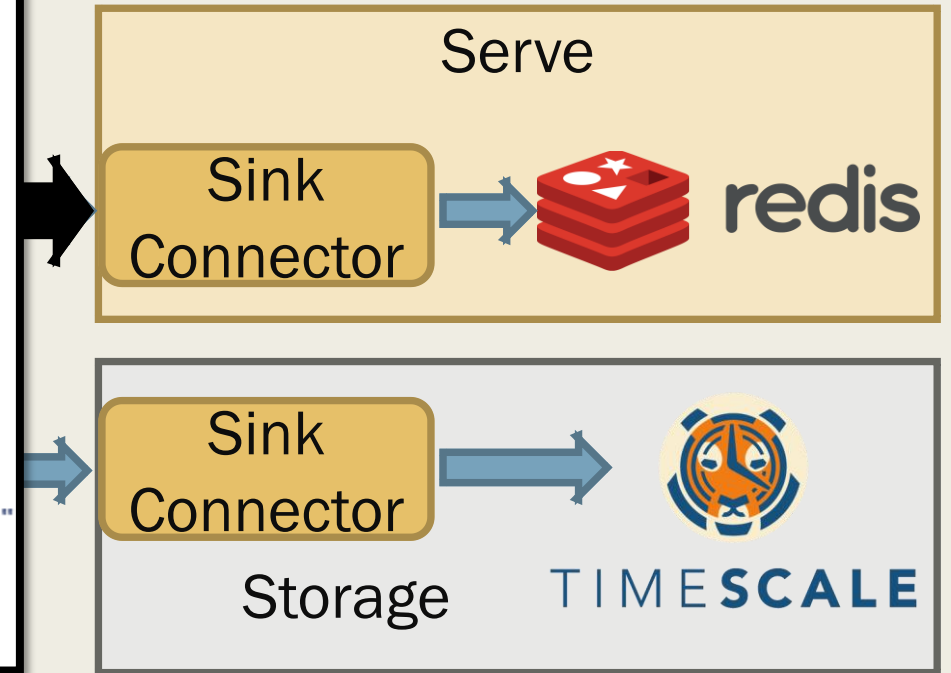
       COUNT(DISTINCT d) AS totalDevices,
       COUNT(DISTINCT CASE WHEN COALESCE(online, FALSE) THEN d END) AS onlineDevices,
       COUNT(DISTINCT CASE WHEN NOT COALESCE(online, FALSE) THEN d END) AS offlineDevices,

       AVG(t) AS avgTemp,
       AVG(h) AS avgHumidity
FROM TABLE(
  HOP(
    TABLE mqtt_enriched, DESCRIPTOR(rt), INTERVAL '10' SECOND,
    INTERVAL '10' MINUTE
  )
)
GROUP BY env, tenant, window_start, window_end;
  
```

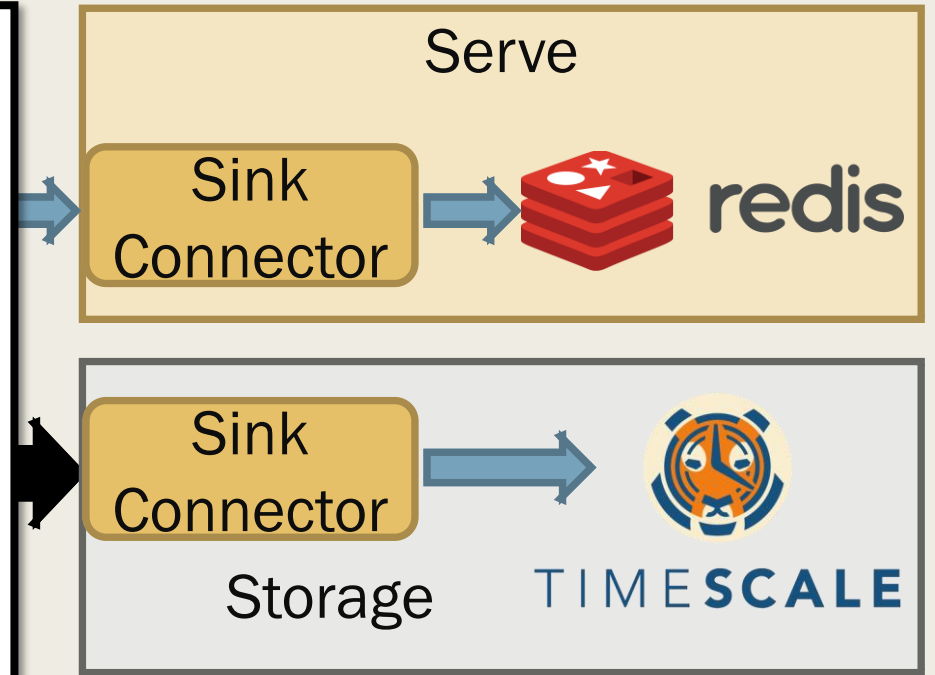
```
{
  "name": "mqtt-analytics-redis-sink",
  "config": {
    "connector.class": "com.redis.kafka.connect.RedisSinkConnector",
    "topics": "recent_summary,low_humidity_streak",

    "redis.uri": "redis://redis:6379",
    "redis.type": "HASH",
    "redis.key.ttl": "600",

    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://kafka-schema-registry:8081"
  }
}
```



```
{
  "name": "sensor-enriched-timescale-db-sink",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSinkConnector",
    "topics": "sensor.enriched.v1",
    "connection.url": "jdbc:postgresql://pvz-db:5432/db",
    "connection.user": "app",
    "connection.password": "app",
    "table.name.format": "sensor_enriched",
  }
}
```



Streaming Big Data from a Particle Detector



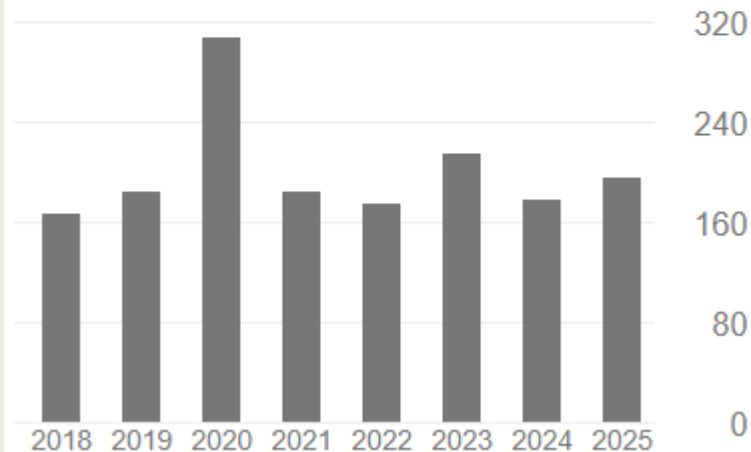
Sergey Kononov

Budker Institute of Nuclear Physics, [Novosibirsk State University](#)

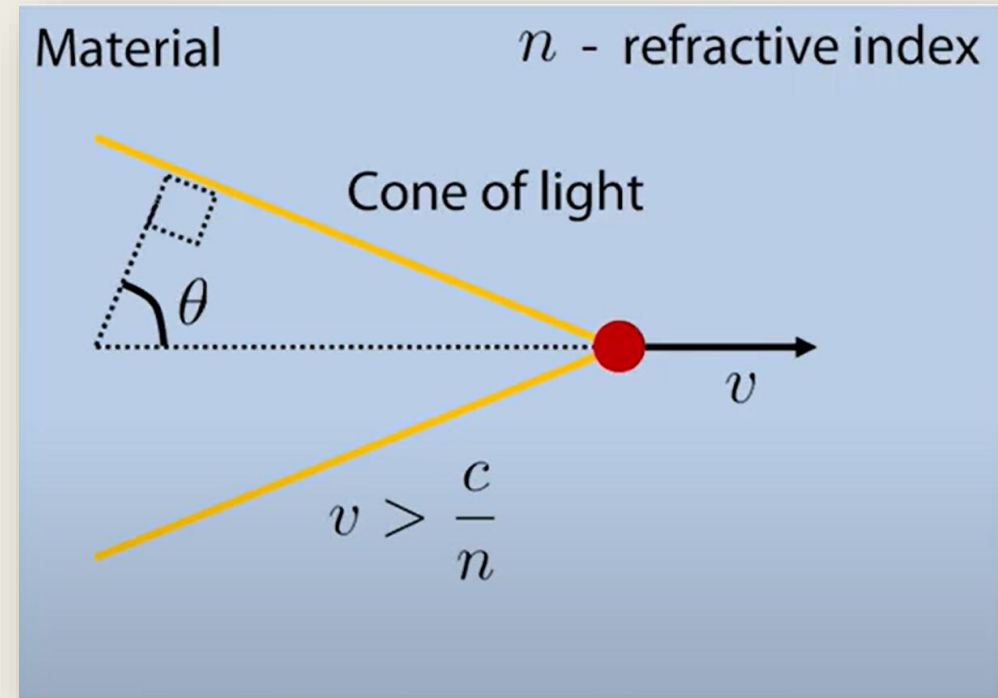
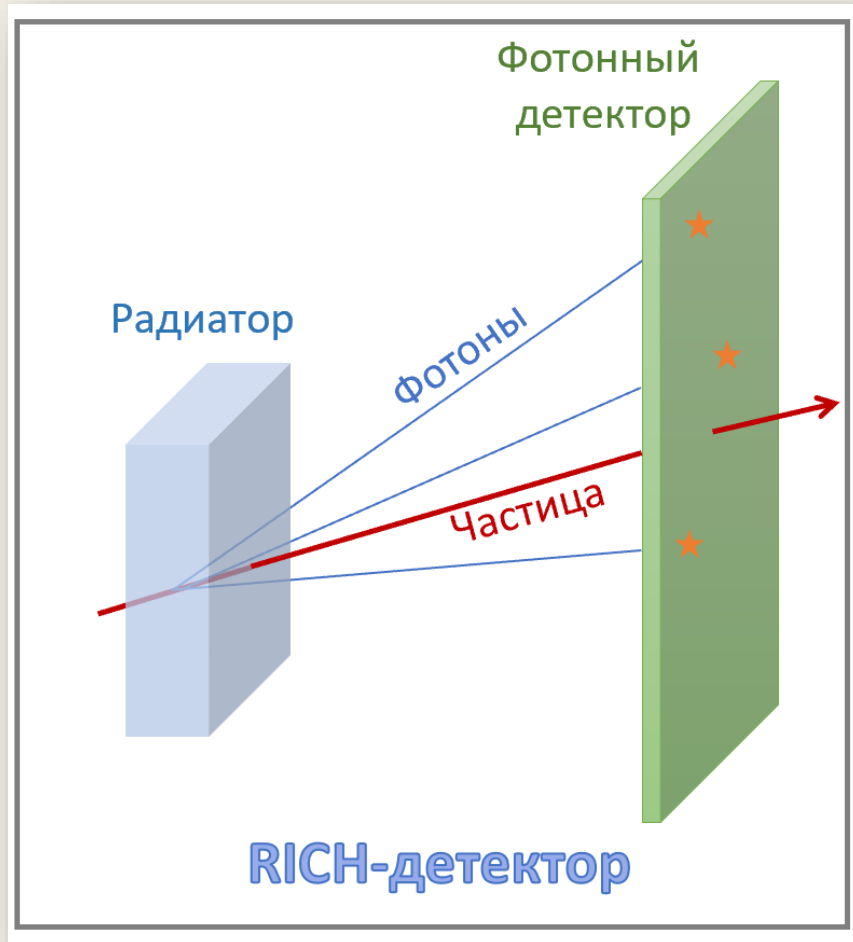
Подтвержден адрес электронной почты в домене nsu.ru

[физика](#) [высокопроизводительные...](#) [наукометрия](#)

Статистика цитирования	3079	1262
h-индекс	32	18
i10-индекс	67	33

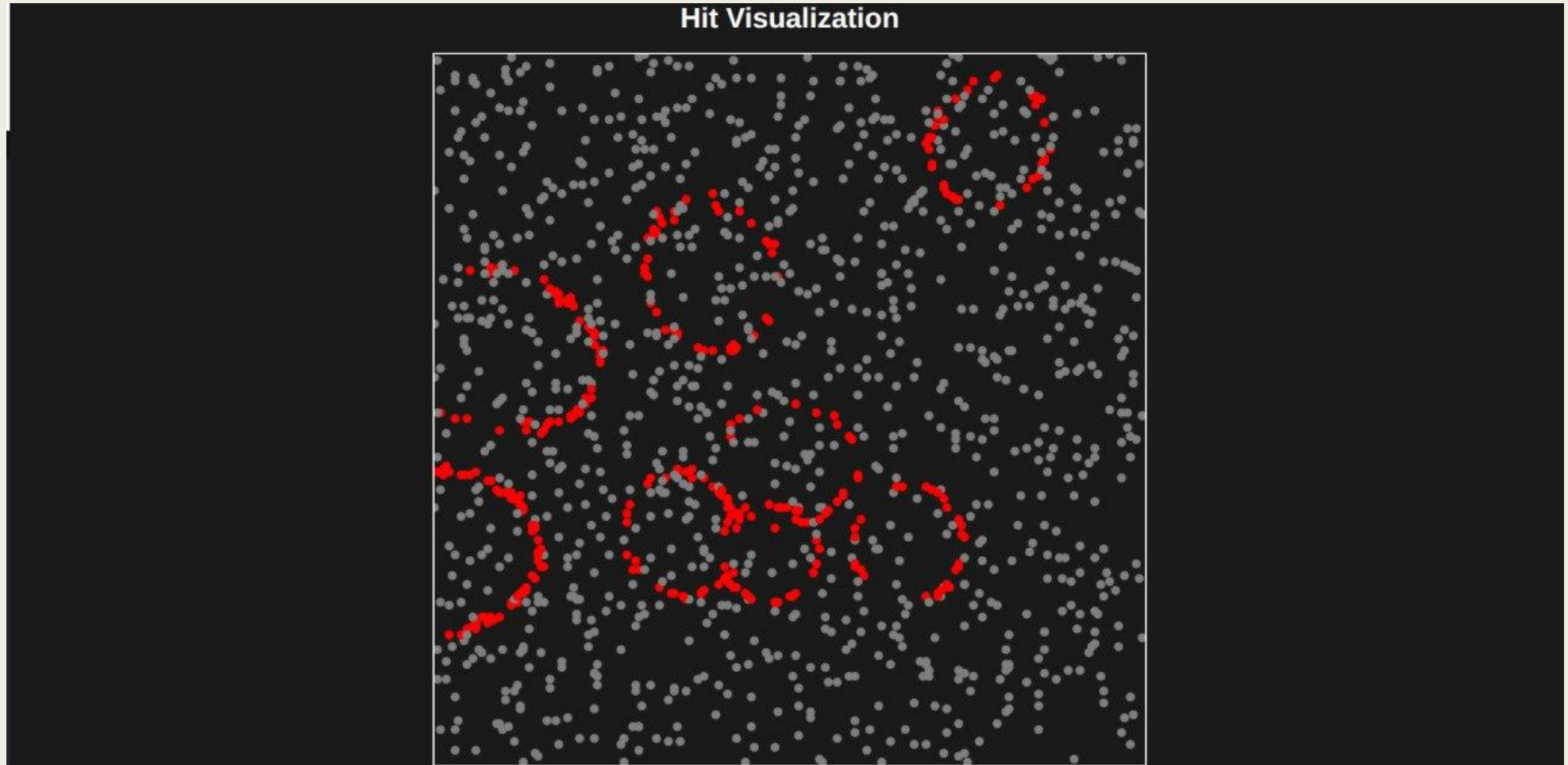


Ring-imaging Cherenkov detector



Cherenkov radiation effect

Demo Cherenkov detector



Почему Cloud?

- CAPEX vs OPEX
- Высокая доступность
- Переносимость (Cloud-first $\neq$ Vendor lock-in)

План релиза (участники)

Сторона	Зона ответственности
Разработка	Склонировать репозиторий, выбрать необходимые компоненты платформы, реализовать бизнес логику аналитики, сконфигурировать (Java-)коннекторы, Docker Compose
Бизнес	Обозначение целей продукта, согласование состава MVP и финального продукта
Эксплуатация / DevOps	Подготовка виртуальной машины, переменные окружения, TLS, сетевые правила, запуск и мониторинг контейнеров

План релиза (цели)

3.1. Продуктовые цели

1. Дать разработчикам готовый шаблон развертывания потоковой платформы.
2. Предоставить воспроизводимый сценарий запуска на виртуальной машине через Docker Compose.
3. Подготовить релиз не как набор исходников, а как продукт: документация, инструкция запуска, security checklist, tutorial, готовая сборка и release-артефакты.

3.2. Функциональные цели

- генерация Docker Compose-конфигурации по тегам конструктора;
- запуск базовой платформы на VM;
- поддержка источника данных через MQTT или Kafka Connect Source Connector;
- доставка событий в Kafka;
- хранение данных в PostgreSQL и/или Iceberg/S3;
- потоковая обработка через Kafka Streams и/или Flink;
- выдача обработанных данных через Redis, PostgreSQL или Trino;
- пример бизнес-логики на синтетических данных датчиков температуры;
- инструкция, как заменить примерную бизнес-логику на бизнес-логику пользователя.

3.3. Нефункциональные цели

- воспроизводимый запуск из чистого клона репозитория;
- запуск без ручной установки Kafka, Flink, PostgreSQL и других сервисов на хост;
- минимизация открытых наружу портов;
- отсутствие отладочных UI-контейнеров в production-профиле;
- конфигурация секретов через `.env` или Docker secrets, а не через исходный код;
- документированный план устранения известных уязвимостей.

План релиза (цели)

3.1. Продуктовые цели

1. Дать разработчикам готовый шаблон развертывания потоковой платформы.
2. Предоставить воспроизводимый сценарий запуска на виртуальной машине через Docker Compose.
3. Подготовить релиз не как набор исходников, а как продукт: документация, инструкция запуска, security checklist, tutorial, готовая сборка и release-артефакты.

3.2. Функциональные цели

- генерация Docker Compose-конфигурации по тегам конструктора;
- запуск базовой платформы на VM;
- поддержка источника данных через MQTT или Kafka Connect Source Connector;
- доставка событий в Kafka;
- хранение данных в PostgreSQL и/или Iceberg/S3;
- потоковая обработка через Kafka Streams и/или Flink;
- выдача обработанных данных через Redis, PostgreSQL или Trino;
- пример бизнес-логики на синтетических данных датчиков температуры;
- инструкция, как заменить примерную бизнес-логику на бизнес-логику пользователя.

3.3. Нефункциональные цели

- воспроизводимый запуск из чистого клона репозитория;
- запуск без ручной установки Kafka, Flink, PostgreSQL и других сервисов на хост;
- минимизация открытых наружу портов;
- отсутствие отладочных UI-контейнеров в production-профиле;
- конфигурация секретов через `.env` или Docker secrets, а не через исходный код;
- документированный план устранения известных уязвимостей.

9.1. Основные угрозы

ID	Угроза	Риск	Где проявляется	План устранения	Приоритет
SEC-01	Публикация внутренних портов наружу	Доступ к Kafka, PostgreSQL, Redis, MinIO, Flink, Kafka Connect из интернета	ports в Docker Compose	Убрать лишние ports , оставить только необходимые entryptoint-порты; использовать внутренние Docker networks	Critical
SEC-02	Отладочные UI в production	Неавторизованное управление Kafka, MQTT, MinIO, Flink или Connect	Kafka UI, MQTT Web, MinIO Console, Flink UI	Не запускать UI-контейнеры в production; вынести их в dev/demo override	Critical
SEC-03	Слабые пароли по умолчанию	Компрометация БД, Redis, MinIO	.env , docker-compose environment	Заменить дефолтные пароли на случайные; добавить .env.example без секретов	Critical
SEC-04	Секреты в репозитории или образах	Утечка доступов	Dockerfile, compose, init scripts	Не хранить секреты в Git; использовать .env , Docker secrets или секреты CI/CD	Critical
SEC-05	Отсутствие TLS для удалённых сервисов	Перехват данных и credentials	Kafka, MQTT, PostgreSQL, Redis, S3/MinIO, Trino	Все удалённые соединения переводить на TLS; plaintext разрешён только внутри закрытой локальной сети VM/demo	High
SEC-06	Запуск контейнеров от root	Повышение последствий при компрометации контейнера	Docker images	По возможности задать non-root user; проверить права volumes	High

SEC-07	Неприкреплённые версии образов	Невоспроизводимые сборки и риск supply chain	<code>image: latest</code>	Зафиксировать версии образов; не использовать <code>latest</code> в release/prod	High
SEC-08	Отсутствие healthcheck/лимитов ресурсов	Сложная диагностика и DoS внутри VM	Docker Compose services	Добавить healthcheck, restart policy, CPU/memory limits	Medium
SEC-09	Открытый Kafka Connect REST API	Возможность создать вредоносный connector	Kafka Connect	Не публиковать REST API наружу; доступ только из internal network; для внешнего доступа — reverse proxy + auth + TLS	Critical
SEC-10	Неограниченные volumes и логи	Переполнение диска VM	Kafka logs, PostgreSQL data, Docker logs	Настроить retention, log rotation, мониторинг свободного места	Medium
SEC-11	Отсутствие резервного копирования	Потеря данных при сбое VM	PostgreSQL, object storage, volumes	Описать backup/restore для volumes и БД	Medium
SEC-12	Отсутствие разделения dev/prod	Dev-инструменты попадают в релиз	Compose-файлы и теги	Разделить <code>docker-compose.dev.yml</code> и <code>docker-compose.prod.yml</code>	High

11. План работ и сроки

Дата	Работа	Ответственный	Результат	Статус
30.04.2026	Подготовить первичный <code>RELEASE.md</code>	Разработка	План релиза в репозитории	Готово
30.04.2026	Заполнить участников релиза и зоны ответственности	Разработка + куратор + DevOps	Таблица участников согласована	Готово
04.05.2026	Подготовить <code>.env.example</code> и обновить <code>.gitignore</code>	Разработка	Секреты вынесены из compose	Готово (необходимо опубликовать <code>.env.example</code> , <code>docker-compose.prod.yml</code>)
04.05.2026	Подготовить <code>docker-compose.prod.yml</code>	DevOps + разработка	Production-like профиль без debug UI	Готово (необходимо опубликовать <code>.env.example</code> , <code>docker-compose.prod.yml</code>)
10.05.2026	Добавить healthcheck/restart policy для сервисов	Разработка + DevOps	Контейнеры диагностируются и перезапускаются	Готово (необходимо опубликовать <code>.env.example</code> , <code>docker-compose.prod.yml</code>)
12.05.2026	Развернуть release candidate на VM	DevOps	Стенд поднят на VM	Запланировано
12.05.2026	Выполнить smoke-тесты и security-тесты	Разработка + DevOps	Протокол проверки	Запланировано
13.05.2026	Согласовать релиз с куратором	Куратор + разработка	Релиз принят к финальной демонстрации	Запланировано
Конец мая 2026	Финальная защита продукта	Вся команда	Демонстрация продукта, документации, сборки, безопасности и кейса использования	Запланировано

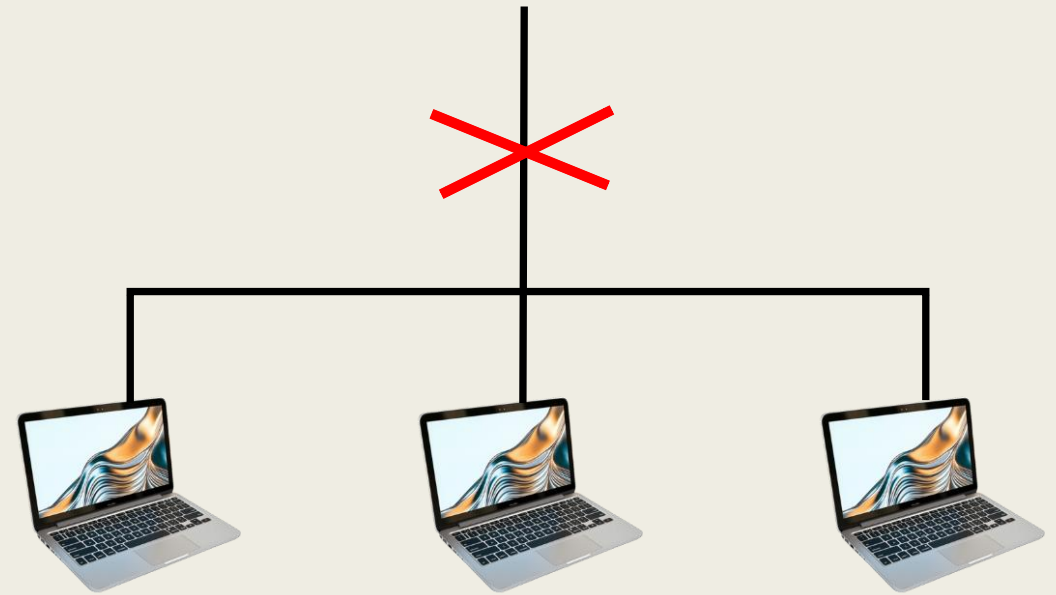
Почему Cloud: CAPEX vs OPEX

Локальный запуск	Запуск в облаке
Закупка железа, сетей заранее	Платим за реально используемые ресурсы
Эксплуатация: место, питание, охлаждение, обслуживание и шум	Эксплуатация: не наша забота



Почему Cloud: высокая доступность

- 2+ зоны
- Резервирование сети и питания
- Балансировщики



Почему Cloud: переносимость

- Если система стала успешной, ничего не мешает развернуть ее локально

